FUTVIZ - FOOTBALL MATCH VISUALIZATION AND PLAYER PERFORMANCE TRACKING USING ARTIFICIAL INTELLIGENCE

Wassaf Ahmed Baloch (02-134222-004)
Murtaza Akram (02-134222-079)
Talha Haider (02-134222-107)

A project report submitted in partial fulfilment of the Requirements for the award of the degree of Bachelor of Science in Computer Science

Department of Computer Science
Bahria University, Karachi
2026

DECLARATION

We hereby declare that this project report is based on our original work except for citations and quotations which have been duly acknowledged. We also declare that it has not been previously and concurrently submitted for any other degree or award at Bahria University or other institutions.

Signature : _____

Name : _____

Reg No. : _____

Signature : _____

Name : _____

Reg No. : _____

Date : _____

APPROVAL FOR SUBMISSION

We certify that this project report entitled **"FUTVIZ - FOOTBALL MATCH VISUALIZATION AND PLAYER PERFORMANCE TRACKING USING ARTIFICIAL INTELLIGENCE"** was prepared by **Wassaf Ahmed Baloch, Murtaza Akram** and **Talha Haider** has met the required standard for submission in partial fulfilment of the requirements for the award of Bachelor of **Computer Science** at Bahria University.

Approved by,

Signature : _____

Supervisor : Miss. Fatima Bashir

Date : _____

ACKNOWLEDGEMENTS

We would like to thank everyone who has contributed to the successful completion of this project. We would like to express our gratitude to our research supervisor, Miss. FATIMA BASHIR for her invaluable advice, guidance and her enormous patience throughout the development of the research.

In addition, we would also like to express our gratitude to our loving parents and friends who have helped and given us encouragement.

FUTVIZ - FOOTBALL MATCH VISUALIZATION AND PLAYER PERFORMANCE TRACKING USING ARTIFICIAL INTELLIGENCE

ABSTRACT

The automation of video analysis in sports has become a crucial tool for any coaches, analysts, or broadcasters that might be looking for performance metrics that provide clear objectivity [1]. This project **FUTVIZ**, offers an advanced system for automated analysis of football matches video footage that is designed using computer vision. With the use of deep learning architectures, this system integrates the **YOLO (You Only Look Once)** object detection framework [2] and the **ByteTrack** [3] multi object tracking algorithm, that not only uniquely identifies the players but then monitors their activity as well. This utility then further extends to the referees and the ball across multiple camera angles and occlusion scenarios. A key innovation put forth by FUTVIZ is the use of its automated **Team Assignment Module**, which uses an unsupervised team classification method using **SigLIP** and **K-means clustering** on the players with the help of their jersey colours. This technology works by first, isolating the torso of a detected player, then introducing bounding boxes, that apply a colour space analysis. The system then distinguishes between the two opposing teams by itself. For computational efficiency, the system uses **batch processing** (for inference) and a **stub caching mechanism** (to preserve tracking data). This will significantly reduce the reprocessing time during the iterative analysis stages of processing. The final output that is generated is a very high yielding, annotated video that will provide real-time visual feedback, including but not limited to team specific markers, a player's unique ID and ball trajectory indicators. Up until now experimental results have demonstrated that the system maintains a high rate of tracking consistently and provides accurate colour based classification. This system was developed using **Python, OpenCV, and Ultralytics**. **FUTVIZ** offers a scalable

base for advanced means of tactical analysis, such as player heatmaps, possession statistics, and formation detection.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

LIST OF APPENDICES

CHAPTER 1

INTRODUCTION

1.1 Background

Sports analytics have been revolutionised by the progress of Artificial Intelligence and computer vision. In Premier League football, insights driven by data are now crucial for tactical improvement and player development [1]. Yet, state-of-the-art automatic systems are often out of reach for many clubs due to the high costs and the need for dedicated equipment. **FUTVIZ** aims to overcome these issues by offering a low-cost Python-based software system, run on a PC, that can take standard match videos to generate professional analysis.

1.2 Problem Statement

Video analysis of matches is time-consuming, inexact and prone to bias. Commercial tracking solutions are that they often require specialised equipment. Also, technical hurdles including moving cameras, obscured players and visibility of the ball may affect tracking. There is a clear need for integrated and cost effective solutions deploying techniques that merge object detection, team recognition and performance calculation in a user-friendly.

1.3 Aims and Objectives

The objectives of the thesis are shown as following:

i) To develop an automatic football visualization pipeline.

ii) To detect the ball, players and referees with YOLO model.

iii) To keep track of all the objects across the video using ByteTrack.

iv) To group players according to their jersey color using SigLIP and K-Means Clustering

v) To use perspective transform and optical flow, to convert pixel values to football pitch dimensions and for camera motion compensation.

vi) Track performance metrics such as speed and distance travelled.

1.4 Scope of Project

FUTVIZ is a PC software that processes football video using a modular processing hosting extracting and tracking players, referees and ball. The system's scope encompasses the use of YOLO [2] for object detection and ByteTrack [3] for maintaining consistent identities across frames, supplemented by K-means clustering and SigLIP [4] to assign players to teams based on jersey colours. The project provides spatial accuracy through the compensation of camera movement using optical flow and perspective transform to convert the spatial coordinates of the pixels to virtual field dimensions. These built-in capabilities enable the computation of quantitative performance indicators such as players' speed and covered distances, leading to the creation of video reports of annotated matches and tabular reports in CSV/JSON formats for tactical and professional analysis.

CHAPTER 2

LITERATURE REVIEW

2.1 Background

The development of football analytics has been shaped by advancements in Artificial Intelligence, computer vision, detecting and understanding football tactical plays and moving to integrate with video. Deep learning models play a critical role in sports analytics, evolving sports analysis from video to data, as seen in advanced systems such as StatsBomb which is referenced in [5] and SkillCorner. Such off-the-shelf systems employ advanced computer vision technologies to deliver event data and physical performance analysis at a high fidelity, but generally require costly equipment and "hi-def" video feeds. This presents a technological gap for many analysts and clubs without substantial financial support.

FUTVIZ seeks to address this issue by providing a low-cost, PC-based platform with a professional-grade offering. The system is based on a Python-based architecture requiring only an off-the-shelf PC with an NVIDIA GPU card for hardware acceleration. By integrating state-of-the-art models such as YOLO [2] for object detection and ByteTrack [3] for consistent multi-object tracking, the system provides a robust foundation for identifying players, referees, and the ball. Also, the context of this project includes overcoming typical technical challenges encountered in sport videography, such as camera motion and distortion, using optical flow and homograph. This integration of computer vision science and sports science allow for pixel data to be translated into performance indicators such as distance and speed

2.2 Comparison with existing study

StatsBomb (recently acquired by Hudl) is a premier event data provider that records over 3,400 events per match, including granular details like "pass footedness" and "pressure". FUTVIZ targets the basic physical performance metrics - namely, speed, distance and tracking - while StatsBomb offers "360 data", which provides additional context regarding a player's actions in critical events, such as shots and passes. StatsBomb is usually reserved for high-end clubs (80%+ of PL) at considerable cost

(pay for play). By contrast, FUTVIZ offers a low-cost, user-friendly, PC-based option that uses automatic (YOLO-based) player tracking to generate these basic tracking metrics without the need for substantial manual verification, or proprietary data sets offered by professional vendors.

SkillCorner is one of the most successful vendors in automated broadcast-based tracking, offering physical performance benchmarks for 120+ leagues. Similarly, to FUTVIZ, SkillCorner takes advantage of single-camera broadcast streams to provide raw player XY coordinates, but with extra functionalities (e.g. "Game Intelligence" when it comes to off-the-ball running and threat creation). Although SkillCorner provides a flexible solution from $499 a month, FUTVIZ is a comparable tracking pipeline (using ByteTrack and optical flow), with no monthly subscription fees and is able to run on commercial hardware. This makes FUTVIZ ideal for semi-professional or grassroots teams needing spatial based analysis without the cost of buying expensive tracking systems.

InStat (also part of the Hudl system) has generally relied on manual event coding and video linking technologies, often offering extensive player fitness and shot maps. However, unlike FUTVIZ, InStat's approach is often "data-on-demand" with video manually coded for accuracy. FUTVIZ advances over the manual processes of "traditional" systems such as InStat by using K-means and SigLip for auto-team labeling, and perspective transformation for coordinate mapping. While InStat caters to large-budget institutional clients, FUTVIZ provides a low-cost fully automated pipeline that includes ball and camera jitter compensation, features that are absent from cheap manual analytics software.

Table 2.1: Comparison with existing study

Feature	StatsBomb	SkillCorner	InStat	FUT VIZ
Ball Tracking	Yes	Yes	No	**Yes**
Camera Correction	Yes	Yes	No	**Yes**
Performance Metrics	Yes	Yes	Yes	**Yes**
Cost-Effective	No	No	No	**Yes**

2.3 Comparison with existing workings

A paper by researchers from the International University of Berlin [3] recently identified some of the key technological building blocks of football analysis. A common choice for many systems is YOLO (You Only Look Once) because of its rapid inference speed and accuracy in player, referee and ball identification. To ensure reliable identification, these detections are sometimes complemented with ByteTrack, a multiple object tracker tracking entity identities and trajectories even during

occlusions by linking high and low-confidence detections. Classification of teams is often done automatically using K-means clustering and SigLip of the colours of the players' clothing. This enables automatic discrimination between opponent teams. To convert video to real-world metrics, researchers apply transformation (homography) to project pixels onto the 2D football pitch plane. Optical flow is also adopted to counteract camera motion for consistent tracking. While high-end systems like StatsBomb [5] and SkillCorner offer unmatched granularity and global benchmarking, they often require significant financial investment and specialized infrastructure. FUTVIZ addresses this issue by developing a fully automated scalable method that works on broadcast or wide angle video. It faithfully emulates key features of commercial systems (i.e. spatial interpolation of the ball and mapping of coordinates into real world positions) at a fraction of the price, opening up the possibility to perform advanced analytics in semi-pro and amateur teams.

2.4 Chapter Summary

This chapter examines the need for automated systems that can be used to drive sports analytics for football clubs and analysts with the need for turnkey football analytics on their PC. It outlines the problems with subjective matches analysis that is time-consuming, subjective, and susceptible to human error, and is not always accurate for performance analysis. The chapter highlights the need for a robust AI-integrated system - FUTVIZ - to overcome these shortcomings by combining object detection, multi-object tracking, and spatial mapping into a scalable solution at affordable costs.

The related work section examines existing research and commercial platforms, noting the effectiveness of the YOLO platform for object detection and ByteTrack for consistency of player movement in case of occlusions. It also reviews the application of K-means clustering and SigLip for automatic team annotation and the need for perspective transformation in translating pixel-based information to realworld dimensions. Though advanced tools are available, there remains a clear need for scalable, PC-based, non-proprietary product that doesn't rely on costly infrastructure.

The chapter finishes with an evaluation of the major competitors (StatsBomb, SkillCorner and InStat) in providing physical analytics, event data (including error-tolerant tracking) and cost-effective solutions to the market. This table (Table 2.1) highlights the different, varied features of current sports technologies and demonstrates how FUTVIZ offers an alternative that is cheaper and more accessible by offering vital tactical feedback such as speed, distance, and heat maps to the user.

CHAPTER 3

REQUIREMENT SPECIFICATIONS

3.1 Existing System

Analysis of football matches is currently subjective and largely manual, with coaches and analysts making visual observations. Typically, the current approach includes coaches scouring through match videos frame by frame, recording the position, movements, and formations of players. This is demanding, taking 4-6 hours for a 90-minute match to analyse, thus reducing the timeliness of feedback and consistency.

Premium systems such as StatsBomb, SkillCorner, and InStat, provide automated tracking and analysis, but they pose considerable challenges [6]. These systems costs between $499 and many thousands of dollars per month, making affordable analysis for semi-professional and amateur teams impossible. Furthermore, many require special cameras, broadcast quality video or a substantial investment to use their technologies.

Current open-source solutions are missing integration between object detection, tracking and classification tools, with users needing to perform manual configuration and connect various tools and platforms. Team classification approaches based on simple colour constraints are prone to skin tone bias and are susceptible to lighting conditions [7]. Multi-object detection and tracking systems tend to fail with crowded penalty areas and occluded players resulting in inaccurate tracking and player ID swaps [3]. Additionally, most approaches lack camera motion compensation and perspective transformation capabilities, affecting the effectiveness of spatial data extraction (such as speed and distance) [3].

Current methods are constrained by human subjectivity and bias in manual analysis, time-consuming processing that cannot extend to multiple matches, affordability for smaller organizations with limited resources, the lack of comprehensive data coverage (either with event or physical metrics, but not both), and the requirement for camera feeds with the right setups or angles. These limitations justify the need for an affordable, automated and integrated system like FUTVIZ.

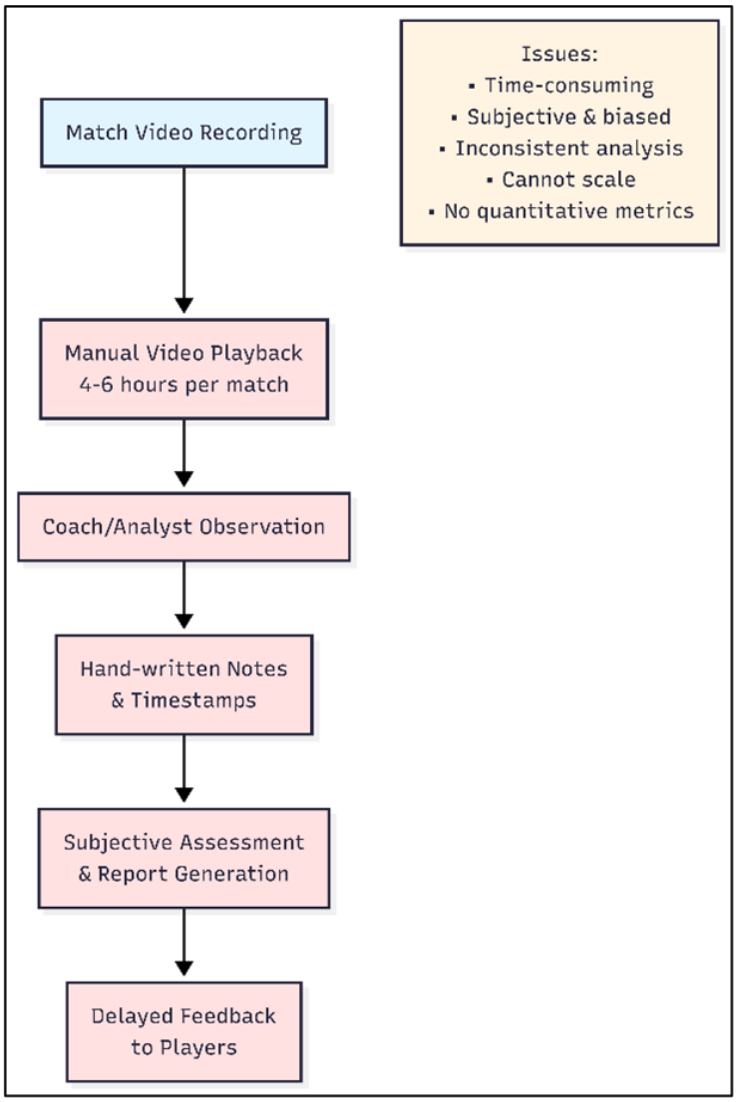


Figure 3.1: Traditional Manual Analysis

3.2 Proposed System

FUTVIZ overcomes shortcomings in current systems through its modular, AI-based system that takes in broadcast-standard video for professional-level analysis. The system is a series of five modules that automate the entire football analysis process.

Module 1: Object Detection and Tracking Leverages YOLO for real-time identification of players, goalies, referees and the ball [2]combined with ByteTrack for track ID persistence across frames [3]. It robustly tracks players across occlusions

and high player density.

Module 2: Team Classification Uses SigLIP vision transformers, UMAP for dimensionality reduction and K-Means clustering to classify players into teams based on jersey colors [8] [3]. The approach is not sensitive to skin colour and is resistant to changes in lighting.

Module 3: Camera Stabilization (Planned) Involves optical flow and homography to compensate for camera movement in order to maintain tracking coordinates and translate from pixels to field dimensions via OpenCV [9]. This enables reliable distance measurements even with moving cameras.

Module 4: Performance Analysis Computes quantitative performance metrics such as speed, distance, heat maps and tactical formations using transformed coordinates. This enables tactical analytics similar to professional tools [5].

Module 5: Visualization and Output Annotates videos with team-coloured markers, player IDs and ball path markers (using Supervision library [7] and exports data in CSV/JSON formats

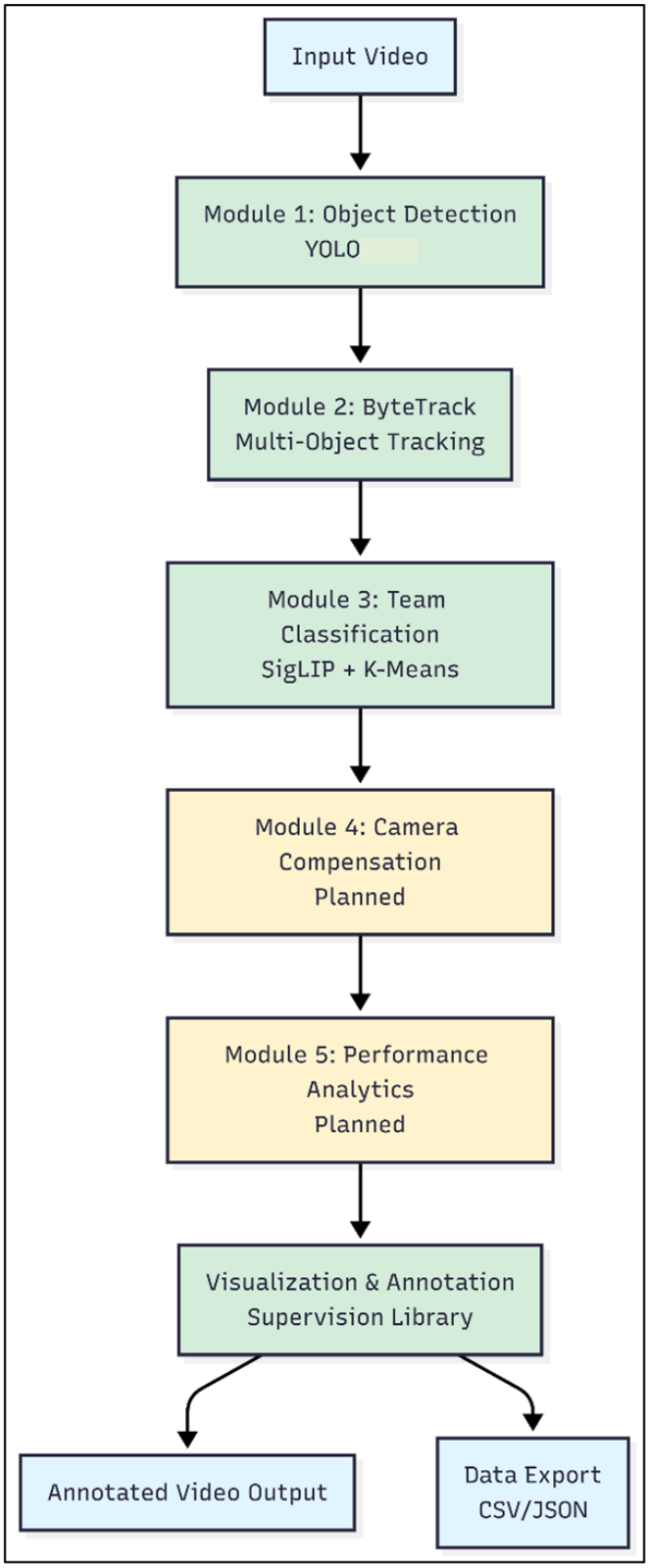


Figure 3.2: Sys architecture Overview 1

3.3 Requirement Specifications

3.3.1 Functional Requirements

FR1: Object Detection. Football players, goalkeepers, referees and ball shall be detected in video using YOLO [2]The system shall allow setting the detection confidence threshold (default 0.1). It shall be able to process a batch of frames.

FR2: Multi-Object Tracking. System shall track objects using ByteTrack [3]. Track IDs to remain consistent across frames, even if track is momentarily lost. System shall maintain a track for at least 30 frames (can be configured)

FR3: Team Classification. System shall automatically sort players into teams. Play shall be done by analyzing jersey colour with SigLIP embedding [8]. System will not be influenced by skin color. At least 100 samples shall be extracted from the video

FR4: Goalkeeper Resolution. System shall identify goalkeepers as a separate class. Goalkeeper team assignment shall be determined by proximity to team centroids

FR5: Annotation and Visualization. System shall draw ellipses around players in team-coloured with Supervision library [7]. Players' IDs shall be color-coded by team. A special marker shall be used to indicate ball location. Real-time annotations shall be performed on video

FR6: Video Output. System shall generate annotated MP4 video files. Output frame rate shall match input video (default: 24 FPS). Video quality shall be preserved without significant compression artifacts

FR7: Configurable Parameters. Users shall be able to specify input/output video paths. YOLO model path shall be configurable. Training data collection stride shall be adjustable. Device selection (CPU/GPU) shall be configurable

Table 3.1: Functional Requirements

ID	**Requirement**	**Description**	**Priority**	**Status**

FR1	Object Detection	Detect players, goalkeepers, referees, ball using YOLOv11x	High	Implemented
FR2	Multi-Object Tracking	Assign and maintain unique IDs using ByteTrack	High	Implemented
FR3	Team Classification	Classify players into teams using SigLIP + K-Means	High	Implemented
FR4	Goalkeeper Resolution	Assign goalkeepers to teams by proximity	Medium	Implemented
FR5	Annotation System	Draw team-colored ellipses, labels, ball markers	High	Implemented
FR6	Video Output	Generate annotated MP4 with preserved quality	High	Implemented
FR7	Configurable Parameters	Allow user configuration of paths and settings	Medium	Implemented
FR8	Homography Transform	Map pixel coordinates to real-world dimensions	High	Planned
FR9	Performance Metrics	Calculate speed, distance, heatmaps	High	Planned

3.3.2 Non-Functional Requirements

NFR1: Performance. System shall process 1 minute of video in under 5 minutes on GPU. Memory usage shall not exceed 8GB during processing. GPU utilization shall remain above 70% during YOLO inference [10]

NFR2: Accuracy. Object detection mAP50 shall exceed 85%. Team classification accuracy shall exceed 90% on test sets. Track ID switches shall be minimized to less than 5% of total tracks

NFR3: Reliability. System shall handle videos of varying resolutions (720p to

4K). System shall gracefully degrade if GPU is unavailable. Error messages shall be clear and actionable

NFR4: Usability. Installation shall be completed in under 30 minutes.System shall run with a single command after configuration. Documentation shall include troubleshooting guide

NFR5: Maintainability. Code shall follow modular architecture principles. Each module shall have comprehensive README documentation. Functions shall include docstrings and type hints

NFR6: Scalability. System shall support batch processing of multiple videos. Processing pipeline shall be parallelizable for future optimization

NFR7: Compatibility. System shall run on Windows, Linux, and macOS. Python 3.8+ shall be supported. CUDA 11.0+ shall be required for GPU acceleration

Table 3.2: Non-Functional Requirements

ID	**Category**	**Requirement**	**Metric**	**Target**
NFR1	Performance	Processing Speed	Minutes per video minute	< 5 min (GPU)
NFR2	Performance	Memory Usage	Peak RAM consumption	< 16 GB
NFR3	Performance	GPU Utilization	GPU usage during inference	> 70%
NFR4	Accuracy	Detection mAP50	Mean Average Precision	> 85%
NFR5	Accuracy	Team Classification	Classification accuracy	> 90%

NFR6	Accuracy	Track Consistency	ID switch rate	< 5%
NFR7	Reliability	Video Support	Resolution range	720p - 4K
NFR8	Reliability	Fallback Mode	CPU operation capability	Yes
NFR9	Usability	Setup Time	Installation duration	< 30 min
NFR10	Usability	Execution	Commands required	Single command
NFR11	Compatibility	OS Support	Operating systems	Win/Linux/macOS
NFR12	Compatibility	Python Version	Version support	3.8+
NFR13	Scalability	Batch Processing	Multiple video support	Yes
NFR14	Maintainability	Documentation	Module READMEs	Complete

3.4 Use Cases

The user-system interaction features two key elements: the User (the Analyst/Coach) and the System (FUTVIZ Pipeline). The main features enable the user to supply raw video of the sport match, set parameters during the processing and receive video of the matches with annotations and export the data.

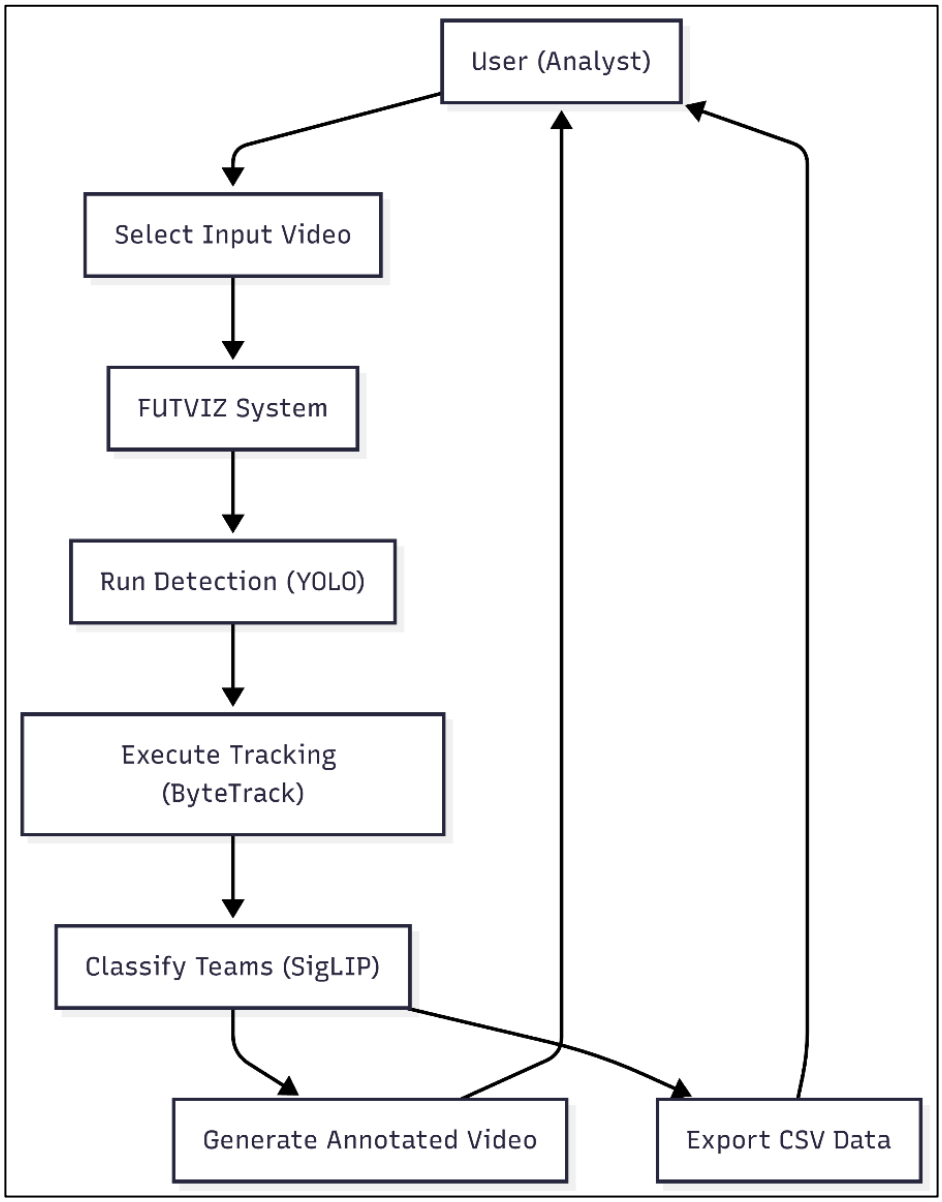


Figure 3.3: Process Match Video

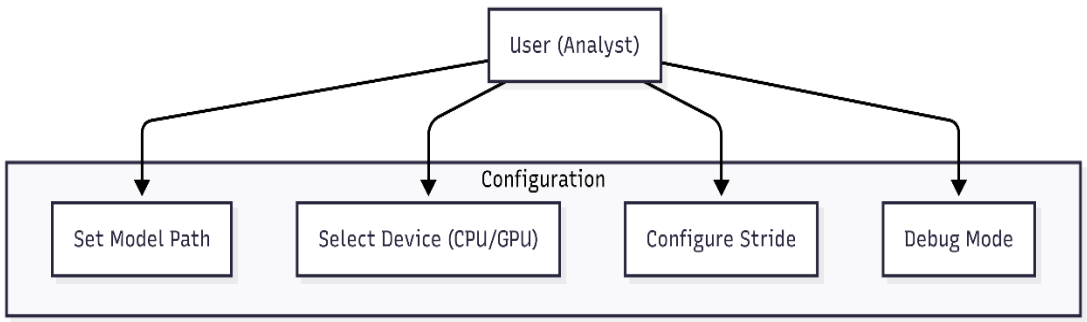


Figure 3.4: System Configuration

Table 3.3: Process Match Video

Use Case ID	**UC-01: Process Match Video**
Primary Actor	User (Analyst)
Pre-condition	System installed, Python environment active, Video file available.
Description	User inputs a raw video file. The system detects objects, tracks entities, classifies teams, and outputs an analyzed video file.
Post-condition	Annotated MP4 file and tracking data (CSV/PKL) are saved to the output directory.

Table 3.4: Configure System

Use Case ID	**UC-02: Configure System**
Use Case Name	Configure Analysis Parameters
Primary Actor	User (Analyst)
Pre-condition	The FUTVIZ system software is installed, and the Python environment is active.
Description	The user adjusts the system settings to personalize the system. This involves the choice of the YOLO model weight file, the device on which to run the analysis (CPU or GPU), and the "stride" value (how often to sample frames for learning which team a player is on).
Trigger	User intends to process a video with non-standard requirements

	(e.g., higher speed or specific custom model).
Basic Flow	1. User opens the configuration file (e.g., main.py or config argument list). 2. User specifies the path to the input video. 3. User sets the model_path (e.g., models/best.pt). 4. User defines the processing device (device=0 for GPU or cpu). 5. User saves the configuration.
Post-condition	The system initializes with the specified parameters for the next execution.

3.5 System Constraints

The design and use of FUTVIZ have hardware, software and environmental constraints that affect its capabilities. With regard to hardware, the system is heavily optimized for using the GPU for acceleration. The system can perform inference on the CPU, but in real-time operations, a CUDA-supported NVIDIA GPU is necessary, as processing high-definition (HD) video with the CPU becomes much slower. Moreover, the system needs to load the YOLO model for object detection and the SigLIP vision transformer model, which consumes a large amount of memory and requires at least 4GB of video random-access memory (VRAM) for minimum requirements and is recommended to have 8GB VRAM for system stability.

In terms of software, the system is bound to particular versions of the main libraries used, such as the Ultralytics for object detection and supervision for tracking. This may cause tracking to become erratic or API errors to arise, requiring careful installation of the Python environment. The pipeline is, therefore, best suited for Linux and Windows environments wherein the NVIDIA drivers and the CUDA toolkit are easily procured, on a system with a graphics processing unit (GPU).

In practice, the system is dependent on the quality of the game video. The computer vision solutions need to see to work; hence, inferior video quality (less than 720p) or blurred motion will make it impossible to spot small objects such as the ball. Also, the automatic team clustering algorithm is dependent visual appearance. If both teams are dressed in the same colors without discernible patterns, then the unsupervised K-Means algorithm may not group them correctly without human intervention.

Table 3.5: SystemConstraints

Category	**Constraint**	**Description**
Hardware	Processing Unit	NVIDIA GPU with CUDA Compute Capability 12.0+ recommended for reasonable inference speeds.
Hardware	System RAM	Minimum 16GB required to handle video frame buffering and track history storage.
Software	Runtime Environment	Python 3.9 or higher is required to support the latest Ultralytics and Supervision libraries.
Software	ML Frameworks	PyTorch (torch) with CUDA support, OpenCV (cv2) for image processing.
Input	Video Format	Standard formats (MP4, AVI, MKV) supported by OpenCV's Video Capture interface.
Input	Resolution	Minimum 720p (1280x720) required for accurate ball and distant player detection.

CHAPTER 4

DESIGN AND METHODOLOGY

4.1 System Architecture Overview

The architecture of the FUTVIZ system is structured as a pipeline that processes video data in multiple steps that first detect objects, then track, classify, and finally analyse [1, 11]. It adopts a sequential feed-forward approach, with objects detected in each frame before being linked to their identities in subsequent frames to gain a comprehensive match analysis. Five subsystems perform key functions in the system. The Input Processor subsystem ingests the input videos and extracts frames in a memory-efficient manner using a generator. The Object Detection Engine uses the YOLO model [12] to detect the players, goalkeepers, referees and the ball with high spatial resolution [2]. This LOG consists of a Multi-Object Tracker component, which employs the ByteTrack algorithm to track objects with ID associations across multiple features [3]. The Team Classification Module is a hybrid AI subsystem that leverages SigLIP Vision Transformers, UMAP dimensionality reduction and K-Means clustering to classify teams based on jersey colors [8]. The Analysis and Visualization Layer aggregates information to determine goalkeeper teams, smoothen tracks with majority voting and annotate the video with computer vision tools [7]

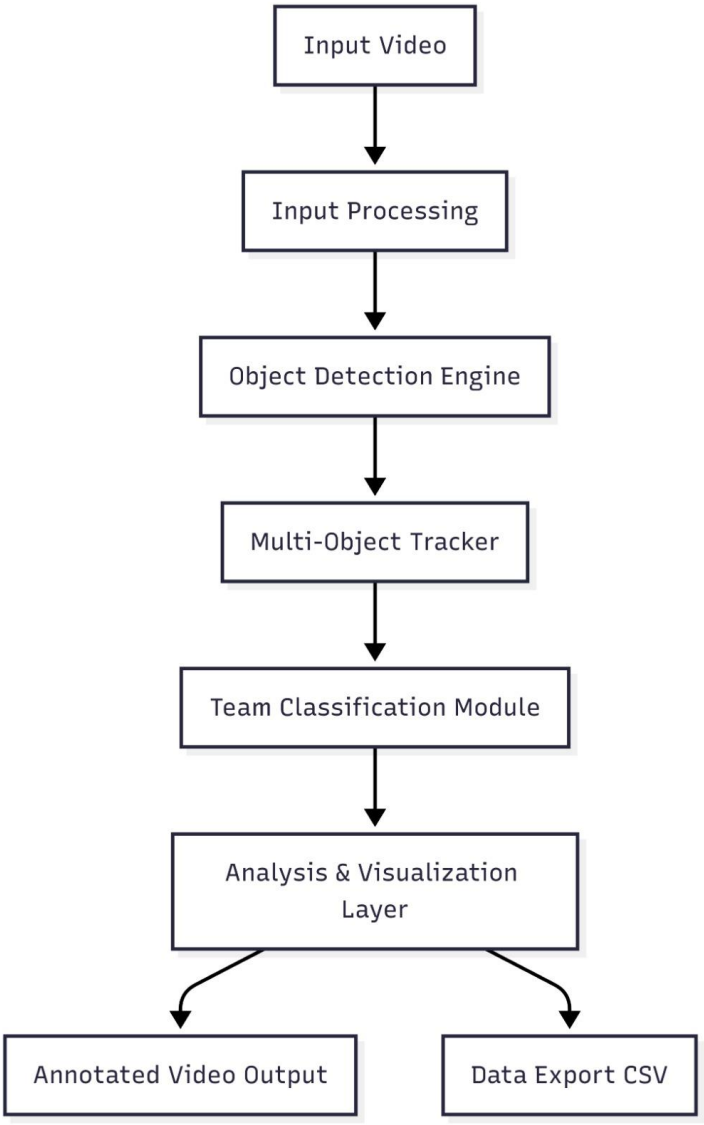


Figure 4.1: Full System Architecture Flow

4.2 Pipeline Design

The system has a three-phase workflow in order to maintain scalability and data integrity. This enables system to learn crucial information, such as team colours or visual identity, before performing match analysis. Phase 1 builds the raw appreciation from the particular video to discern the environment. Phase 2 utilizes that data to train the unsupervised learning models that distinguish between the two teams [7]. Phase 3 executes the full inference pipeline, applying the learned models to every frame of the video to generate the final analysis.

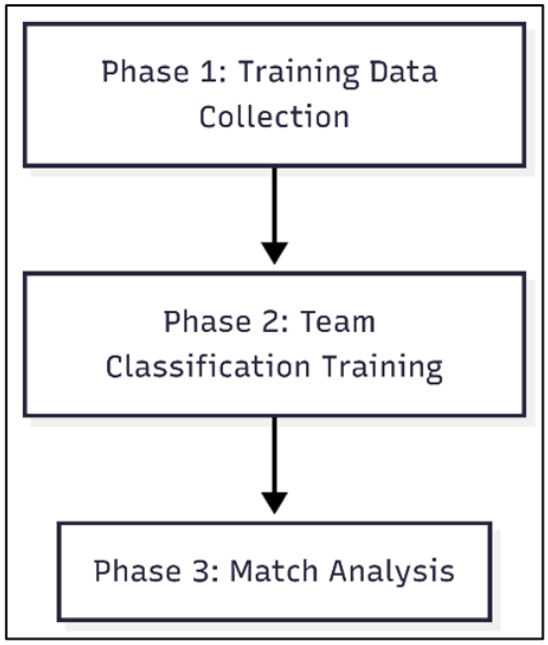


Figure 4.2: Three-Phase Pipeline Design 1

4.2.1 Phase 1: Training Data Collection

Before analysis begins, the system must learn the visual appearance of the teams in the specific match video. This phase utilizes the TrainingDataCollector module to gather representative samples of player jerseys [11]. Instead of processing every single frame, which would be computationally expensive for a training step, the system iterates through the video using a configurable stride parameter, which defaults to 20 frames. For each sampled frame, the YOLO model detects objects [13], and the system extracts cropped images of detected players. To ensure high-quality training data, a stricter confidence threshold of 0.3 is applied during this phase to avoid including blurred or partial detections in the training set.

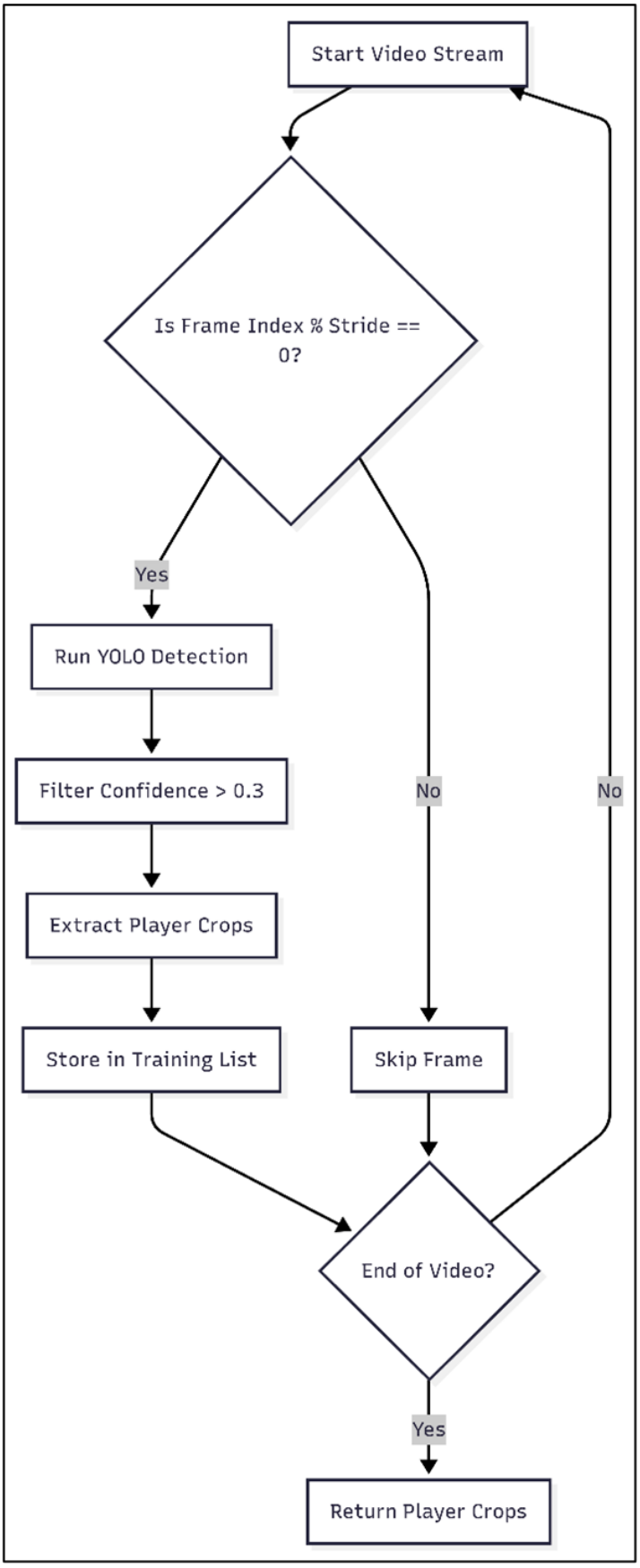


Figure 4.3: Training Data Collection Workflow

4.2.2 Phase 2: Team Classification Trainig

In this phase, the extracted player crops are processed by the TeamAssigner to build a color model for the match. The SigLIP model first processes the player crops to generate high-dimensional semantic embeddings, converting the visual image into a mathematical vector [8]. These 768-dimensional embeddings are then reduced to a 3-dimensional manifold using UMAP, making the data suitable for clustering. The K-Means algorithm partitions this reduced data into two distinct clusters representing the two teams. Finally, the system calculates the centroid of each cluster and extracts the dominant jersey color using a center-patch analysis technique to label the teams visually [3].

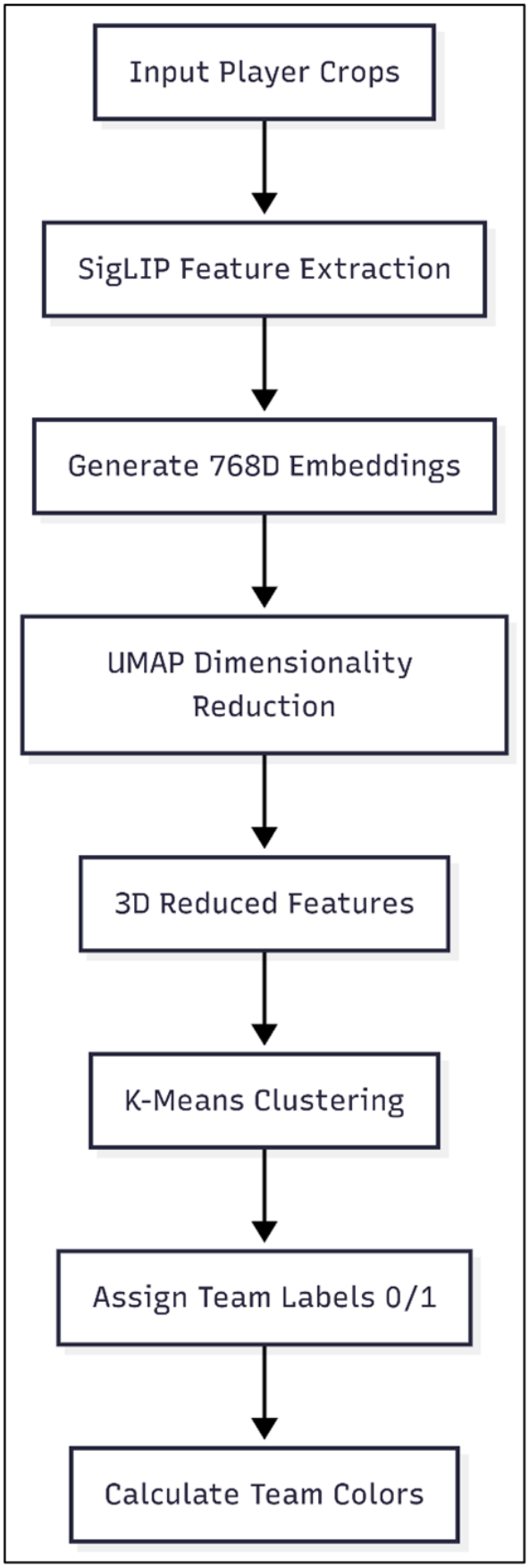


Figure 4.3: Team Classification Training Pipeline

4.2.3 Phase 3: Match Analysis

In the last phase, the inference pipeline is run on the video. The Match Analyzer performs the analysis on a frame by frame basis. It starts by loading the object tracks for the current video frame (players, ball, and referees) [2]. It then inputs the players' crops into the pre trained SigLIP model to predict their teams based on the training in Phase 2. A majority voting (5 frames) is used to avoid flickering (player ID changing from one team to the other quickly). The final decision on the player's team is made only if the team is predicted consistently. To assign the goalkeepers to a team, it is not done based on their jersey color, but based on where they are on the field with respect to the centroid of the players of the team [5].

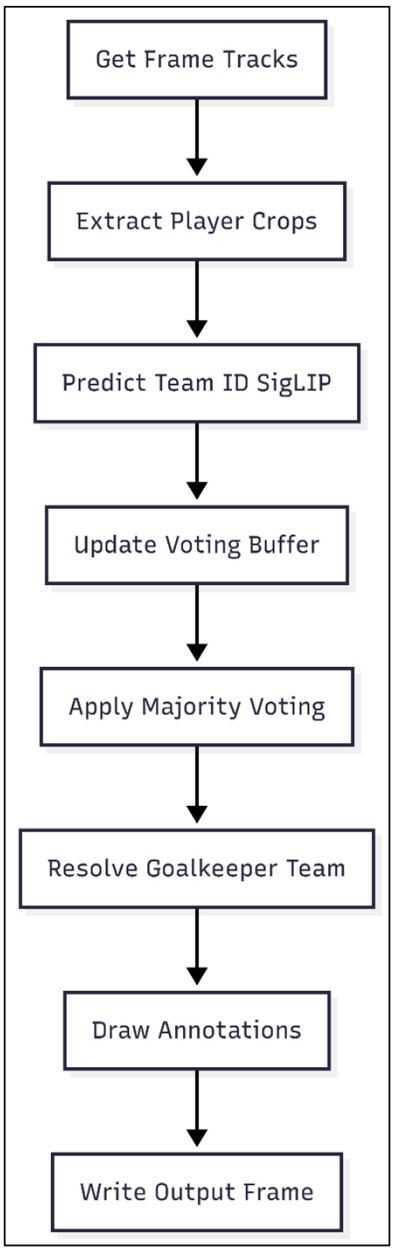


Figure 4.4: Match Analysis Processing Flow

4.3 Object Detection Module Design

The Object Detection Module is the first step of the FUTVIZ system, tasked with identifying and classifying objects in each frame of the video. This component takes raw images from the input module and applies the YOLO neural network to detect the coordinates of the objects present in the image and their confidence in belonging to a specific class. Our design factors in the unique demands of football video, including changes in scale, high velocity and collision between players. Separating detection from tracking allows for a pure tracker to have quality inputs.

4.3.1 YOLO Model Selection

Choosing which YOLO architecture to use was a key design choice balancing performance speed and accuracy. This system uses the YOLOv26 model (the Extra Large model size, rather than the Nano and Small models generally used for mobile applications. Although the Nano version of the model has lower inference times, suitable for processing on mobile devices, it cannot detect small objects, such as the ball or players in the far distance in broadcast shots. The YOLO26 variant has the generation and number of parameters required to achieve high recall rates for these smaller objects, meaning that the ball tracking system works even during long passes [14]

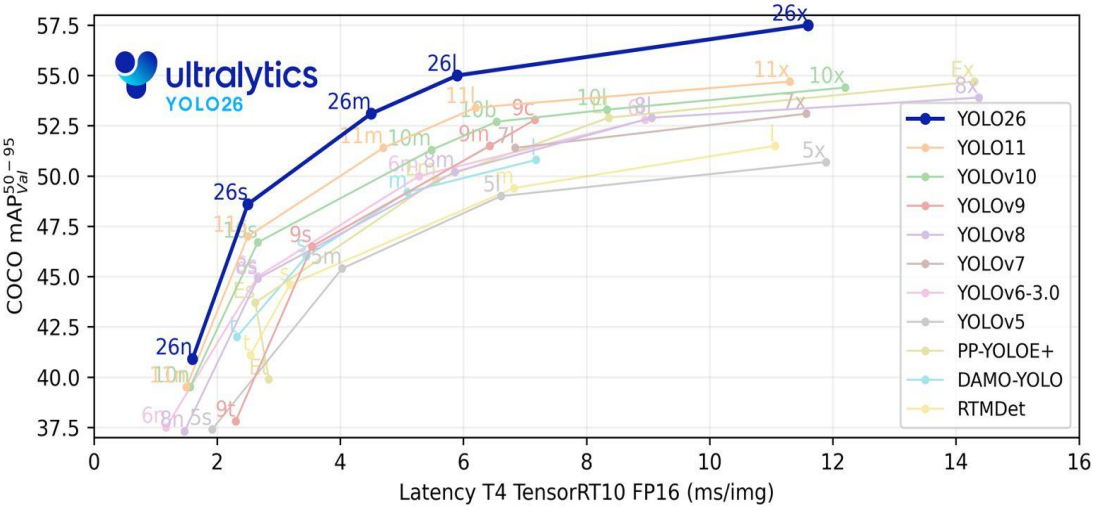


Figure 4.6: YOLO Model Variants Comparison

4.3.2 Custom Dataset Preparation

In order to train a system suitable to the visual conditions of football matches, the proposed dataset strategy identifies four classes: Ball, Goalkeeper, Player and Referee. The distinction between the classes is vital for the system's workflow: these different types of entities are subject to different rules during analysis. For example, goalkeepers need to be separated from players to prevent team kits mixing with goalkeepers. The data preparation process involves the labelling of thousands of

images to include these classes under a variety of different lighting and camera conditions [7].

Table 4.1: Dataset Class Distribution

Class ID	Class Name	Usage in System	Detection Difficulty
0	Ball	Trajectory tracking	Very High (Small size)
1	Goalkeeper	Team assignment logic	Medium
2	Player	Clustering & Speed	Low
3	Referee	Exclusion from stats	Medium

4.3.3 Model Training Strategy

The training approach involves transfer learning as it will train a pre-trained YOLO model to suit the football domain. It follows the steps of recalling weights that have been trained on the COCO dataset as this gives a solid interpretation of the overall characteristics such as edges and forms. The model is then shaped on the given football dataset. In this fine-tuning, the system optimizes the loss functions over the four target classes. An important part of the strategy is that the lower confidence threshold (0.1) is used when making an inference through the strategy, as opposed to training, where a higher confidence threshold is used, since Recall should be prioritized over avoiding relevant objects instead of just being filtered by the following tracker, that is, false positives. [2].

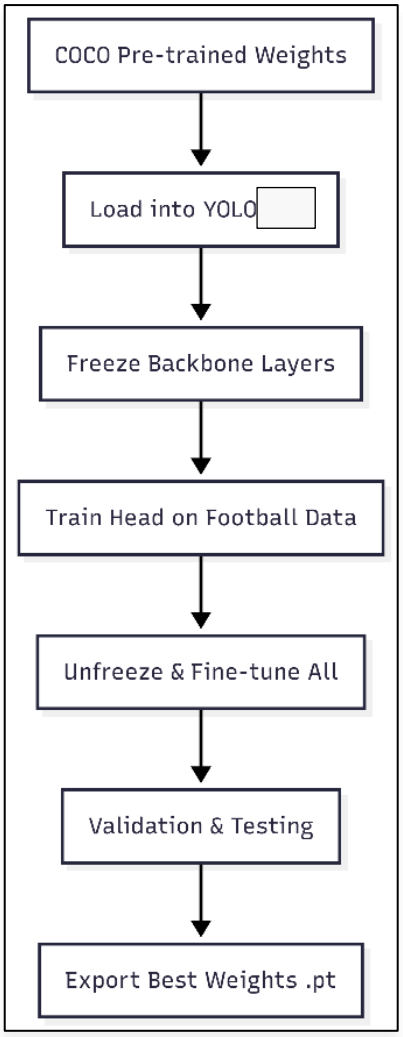


Figure 4.7: Transfer Learning Strategy Diagram

4.4 Multi-Object Tracking Design

Multi-Object Tracking design aims at tracking the identity of players through time, which is not simple because of the high number of occlusions as well as non-linear movement patterns in football. The system provides a state based tracking system which assigns each identified object to a state among a number of states: New, Tracked, Lost or Removed. This state machine allows making sure that temporary detection failures, like a player going behind a referee do not necessarily directly lead to the generation of a new invalid ID. [3]

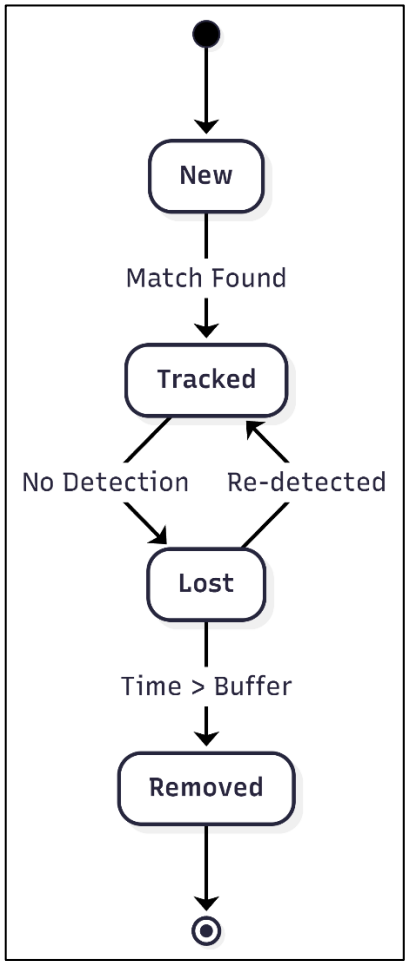


Figure 4.8: Multi-Object Tracking State Diagram

4.4.1 ByteTrack Integration

It incorporates into the system the ByteTrack algorithm that stands out as it uses low-confidence detections, which would be discarded by other trackers. Valid player detections are sometimes lower in confidence score in crowded penalty areas as a result of occlusion. ByteTrack tries to match high-confidence detections initially and a second matching round with the other tracks with these less-confidence boxes. This two-step identification step will greatly decrease the amount of fragmented tracks and ID switches. [3].

Table 4.2: ByteTrack Configuration Parameters

Parameter	**Value**	**Description**
Track Activation Threshold	0.25	Minimum confidence to start a new track
Lost Track Buffer	30	Frames to keep a lost track alive (approx 1.2s)
Match Threshold	0.8	IoU requirement for associating detection to track
Minimum Consecutive Frames	3	Prevents noise from creating instantaneous tracks

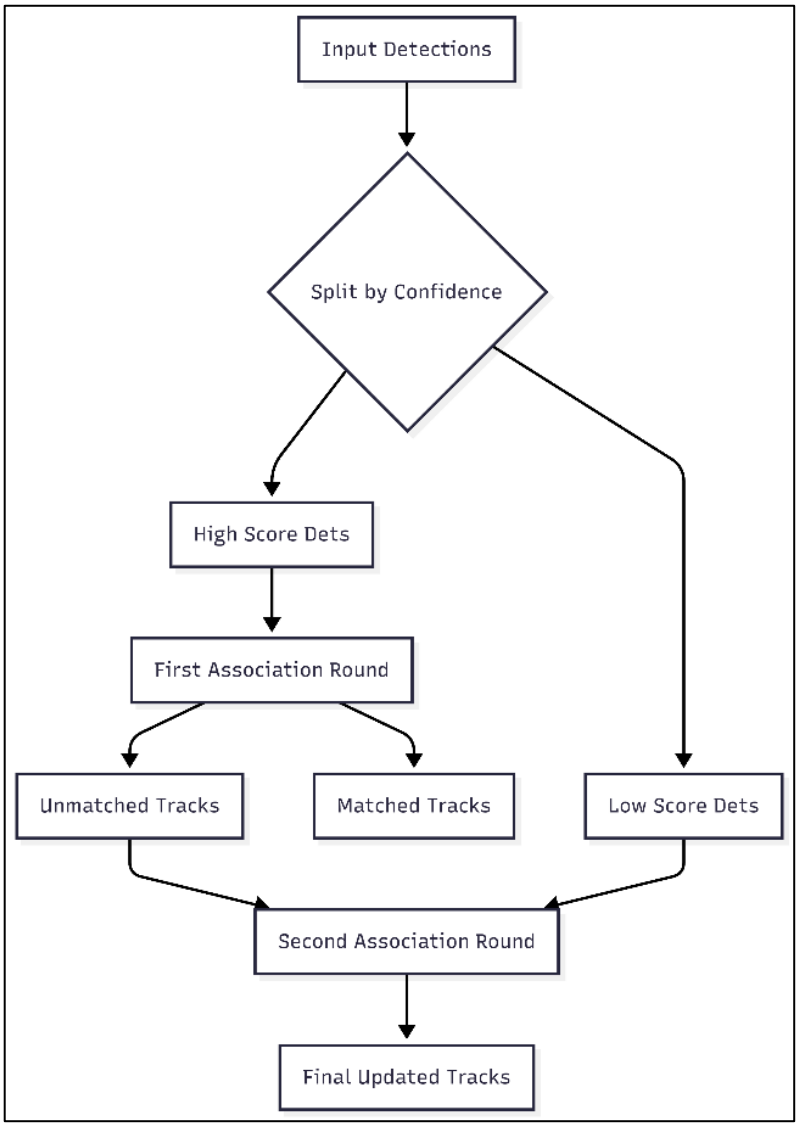


Figure 4.9: ByteTrack Algorithm Flow

4.4.2 Track Persistence Strategy

To cope with brief absences strongly, the system uses track persistence strategy, which is determined by a buffer window which has a configurable value. A tracked object is in a Lost state once no longer tracked and is not automatically deleted. The system anticipates the position of the object by a Kalman Filter and proceeds to search the object to a maximum of 30 frames. In the event that the object recurs in this window and the location is identical to what it was predicted, the original ID is reinstated. This keeps continuity in the game when the players are out of the camera shot in a short period or are obstructed by other players.

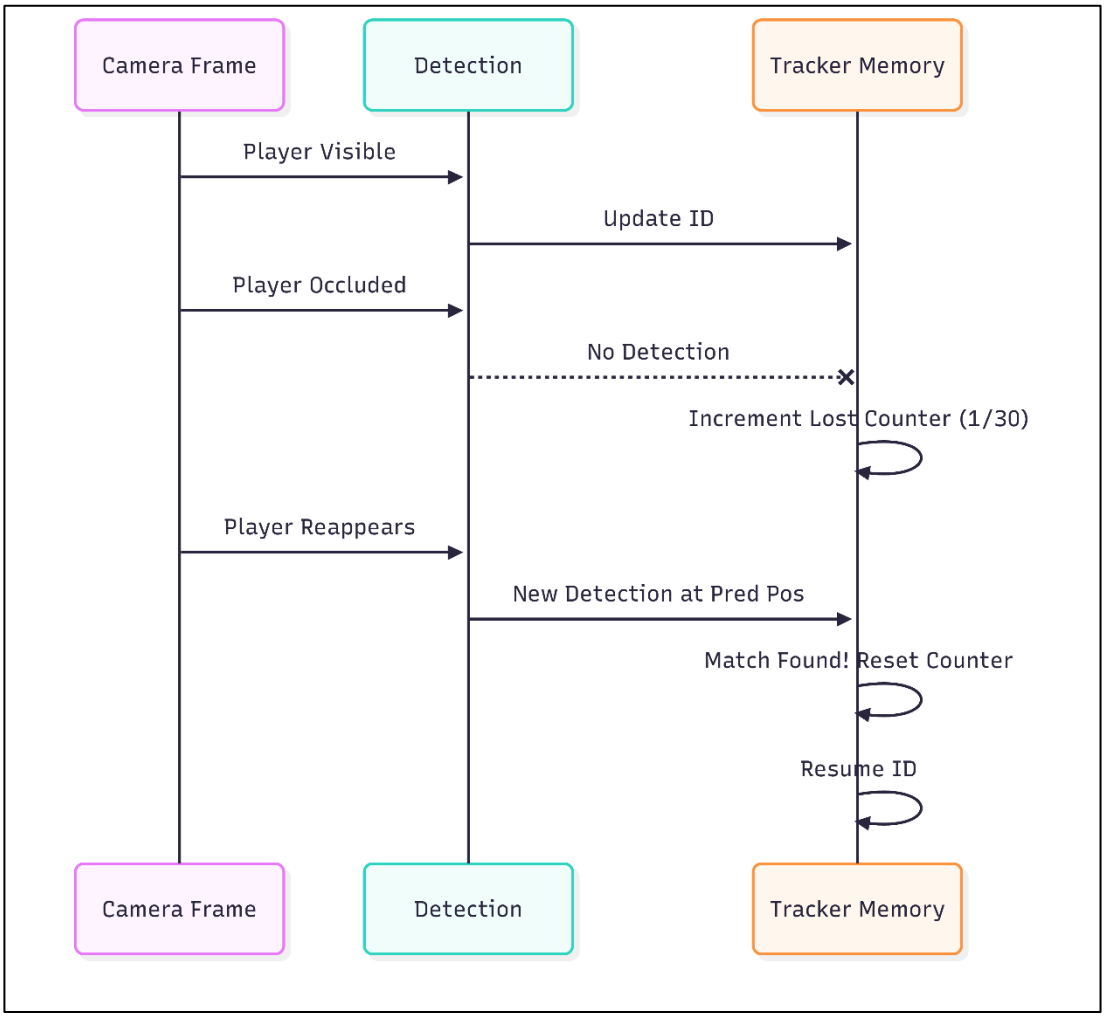


Figure 4.10: Track ID Persistence Mechanism

4.5 Team Classification Design

The Team Classification module resolves the issue of identifying the same classes of players according to the team in which they play. This design uses a semantic interpretation of player appearance, unlike the traditional methods which use simple color histograms, which tend to break down under different lighting conditions. The workflow is used to extract visual features of player crops, project them to a manageable dimensionality then clusters the features to autonomously detect the two separate teams without any manual team color inputs. [8]

Figure 4.11: Team Classification Complete Workflow

4.5.1 SigLIP Embedding Extraction

The SigLIP Vision Transformer is a model trained on large image-text datasets that are used as the feature extraction operator. The model produces a 768-dimensional embedding of a cropped image of a player when the image is passed through SigLIP. This is a vector that represents the semantic meaning of the picture, the details of a jersey design and color without paying attention to the insignificant objects, such as a grass in the background or the skin tone of the player. This produces a powerful descriptor which clusters together players of a team in the feature space despite their pose or lighting. [8]

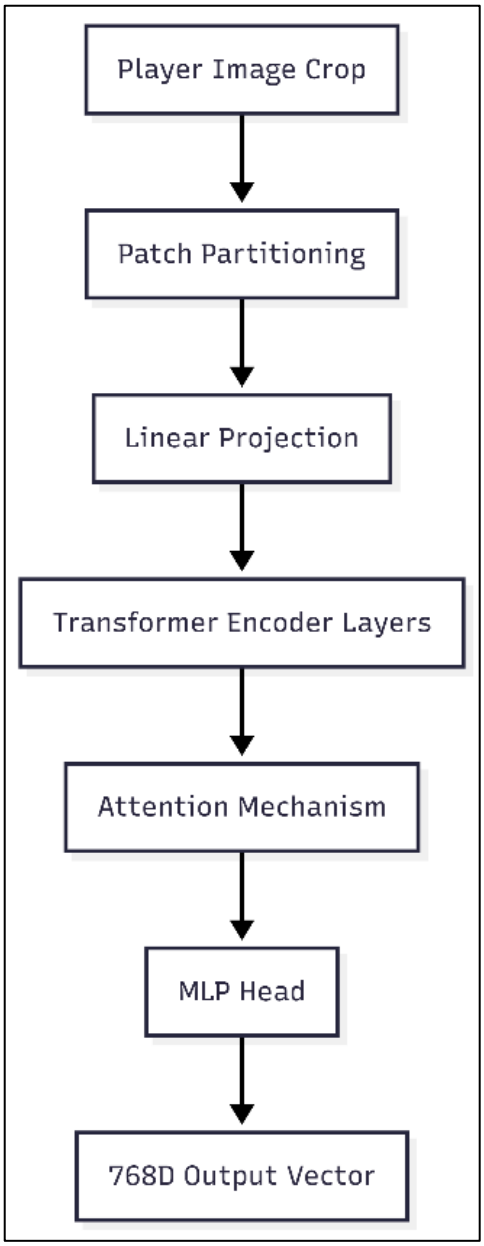


Figure 4.12: SigLIP Vision Transformer Architecture

4.5.2 UMAP Dimensionality Reduction

Processing 768-dimensional data is computationally costly and this is susceptible to the curse of dimensionality where distance measures lose significance. To alleviate this, the system uses Uniform Manifold Approximation and Projection (UMAP). This method projects the high-dimensional embedding's onto a 3 dimensional manifold. In contrast to linear algorithms, such as PCA, UMAP does not distort the local and global structure of the data, meaning that the different clusters of the two teams in the high-dimensional space can be separated in the lower-dimensional representation..

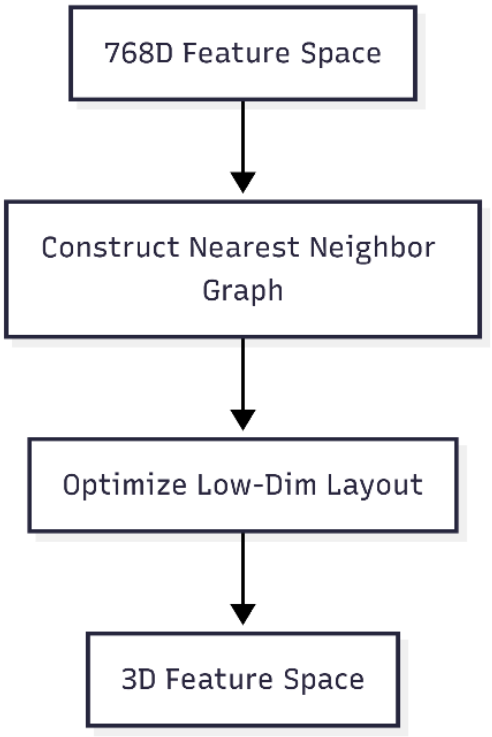


Figure 4.13: UMAP Dimensionality Reduction Concept (768D → 3D)

4.5.3 K-Means Clustering

After the information is brought down to 3 dimensions, the K-Means clustering algorithms enables the system to automatically divide the players into 2 teams. The algorithm is initialized with a value of K=2 which compels the data points into two separate groups. It will continually refine the coordinates of two centroids until the variance in each cluster gets minimized. This yields a binary classification (0 or 1) of each player crop in the training set, which is the ground truth of which visual features are associated with which team.

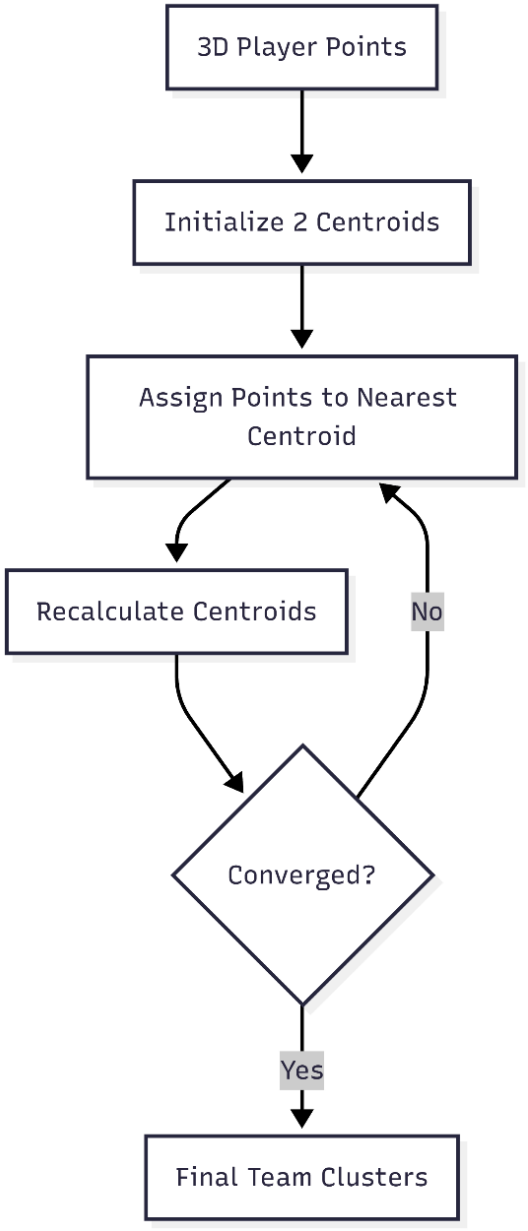


Figure 4.14: K-Means Clustering on Reduced Embeddings

4.5.4 Robust Color Extraction

The system needs to extract some representative color of each team cluster and visualize the team identity on the scoreboard and annotations. An average of the overall player picture usually causes smuddy colors because they incorporate green grass, or skin color. The scheme uses a very strong Center-Patch strategy of extraction and then the central part of the binding box is checked. There is the highest likelihood of the jersey of a player to be in this area. The median colour of this patch is computed to remove pixel noise and shadows to give a clean, recognizable colour to visualise..

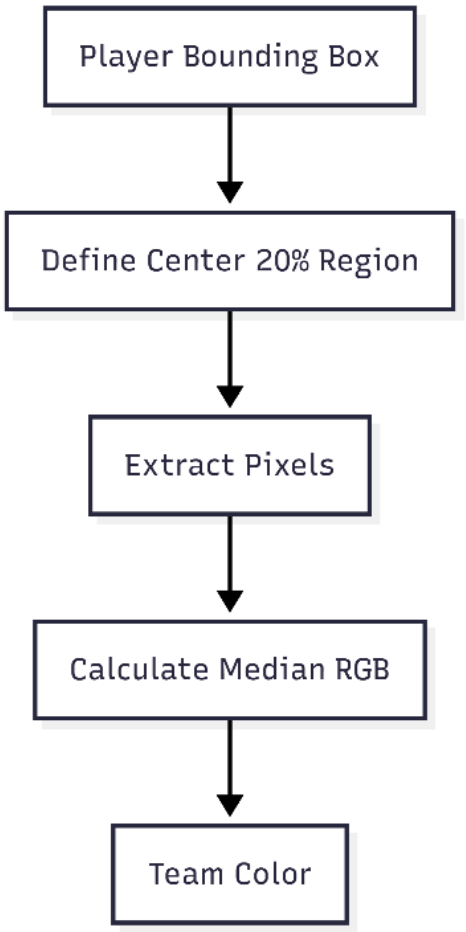


Figure 4.15: Center-Patch Color Extraction Strategy

Table 4.3: Color Extraction Algorithm Comparison

Algorithm	Pros	Cons	Selected for FUTVIZ
Center-Patch Median	Robust to background & skin	Ignores shorts/socks	Yes
Full Image Average	Simple to implement	Muddy colors (mixes grass)	No
K-Means on Pixels	Accurate dominant color	Slow per-frame processing	No
Histogram Peak	Good for solid kits	Fails on striped kits	No

4.6 Spatial Transformation Design

This module aims to convert the distorted broadcast video perspective into a flat, top-down coordinate system for accurate tactical analysis [9]

4.6.1 Homography Fundamentals

Homography utilizes a 3x3 transformation matrix to mathematically map pixel coordinates from the camera view to real-world field meters [9]

4.6.2 Keypoint Detection with YOLOv26-pose

A customised YOLOv26-pose model recognises pitch-specific objects (penalty and corner flags) which provide the reference points used in this homography calculation.

4.6.3 Top-Down View Generation

The system applies the calculated homography matrix to the bottom-center coordinate of every player bounding box to project their position onto a virtual 2D tactical board.

4.7 Performance Metrics Module (Planned)

This module derives quantitative physical data from the transformed coordinates to provide actionable insights into player performance [6]

4.7.1 Speed Calculation

Player speed is calculated by measuring the real-world distance traveled between consecutive frames divided by the time duration of a single frame [6]

4.7.2 Distance Tracking

Total distance covered is computed by accumulating the frame-to-frame displacement vectors for each unique track ID throughout the duration of their appearance.

4.7.3 Heatmap Generation

Spatial heatmaps are generated by aggregating the transformed coordinates of a player over the match duration into a 2D density grid, highlighting their primary areas of operation.

4.8 Chapter Summary

Chapter 4 featured the comprehensive architectural design and approach scheme of the FUTVIZ system, which established it as a modular, feed-forward pipeline of five main subsystems, namely, Input Processing, Object Detection, Multi-Object Tracking, Team Classification, and Analysis. A strategic division of workflow is made with the help of three operational phases of the workflow, viz., Training Data Collection, Team Classification Training and Match Analysis as the key to scalability and making sure that unsupervised learning of team visual signatures will be performed successfully before the full inference.

The chapter elaborated the technical argument that underpinned the choice of YOLOv26 to form the Object Detection Module, which focused more on high recall to incorporate small objects such as the ball that are present and visible in wide-angle broadcasts. It also described how the ByteTrack algorithm is incorporated into the Multi-Object Tracking Module that uses a dual-matching methodology to treat occlusions and achieve related identities among the entities in the match. In Team Classification, the design goes beyond the classical pixel-based frameworks by using SigLIP Vision Transformers to extract semantic features followed by dimensionality reduction with Uniformed Map Analysis and K-Means clustering to classify teams well across any light condition and no matter the skin tone.

Lastly, the chapter described how the Spatial Transformation and Performance Metrics modules were to be designed. These components of the future will be based on homography matrices with syndication and the key point detection aspect of its operation through the YOLAB. Both of these initial pieces of work enable the next step in more sophisticated tactical analytics, such as calculating player speed, total distance covered and creating spatial heat maps, which is the goal of the project to provide professional quality insights on available hardware.

CHAPTER 5

SYSTEM IMPLEMENTATION

5.1 Broadcast View Module

The Broadcast View Module represents the visual output of the FUTVIZ pipeline. Its main feature is the use of overlays that are applied to the original two-dimensional broadcast video stream. By parsing each frame, it consumes the coordinates, IDs, and team tags obtained from YOLO tracker and SigLIP classifier. This way, it attaches the visual elements that serve as a reference point to the objects: team-colored ellipses under players' feet, tracking ID numbers, dynamic highlights for the football, and the referee. For the sake of smoothness, the module uses the stabilized coordinates from the optical flow pipeline.

The drawing primitives provided by Roboflow Supervision are essential due to their optimized way of drawing clean geometric shapes over raw arrays. YOLO26x and ByteTrack are responsible for providing the locational tensors and integer IDs that help in attaching graphics to objects. OpenCV is used for image processing, overlay rendering, and exporting the output in H.264 MP4 format.

The module represents a promising tool for analyzing match performance for coaches and analysts as it allows not only to determine players' teams but also track their off-ball actions and see the current position of the ball.

Figure 5.1: Broadcast View with speed and possession

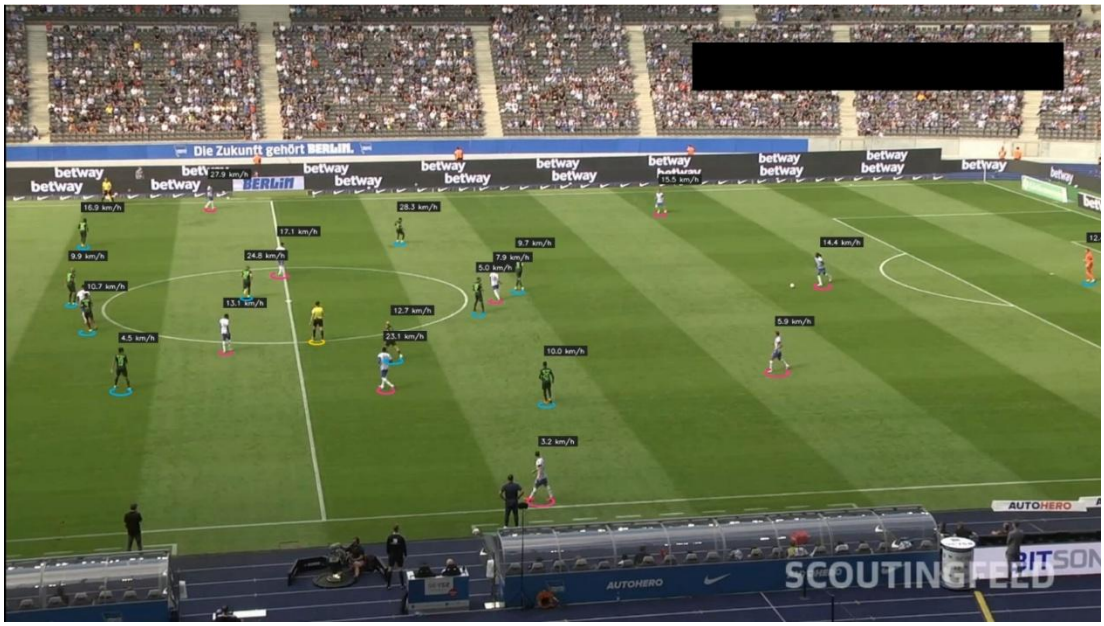


Figure 5.2: Broadcast View with speed

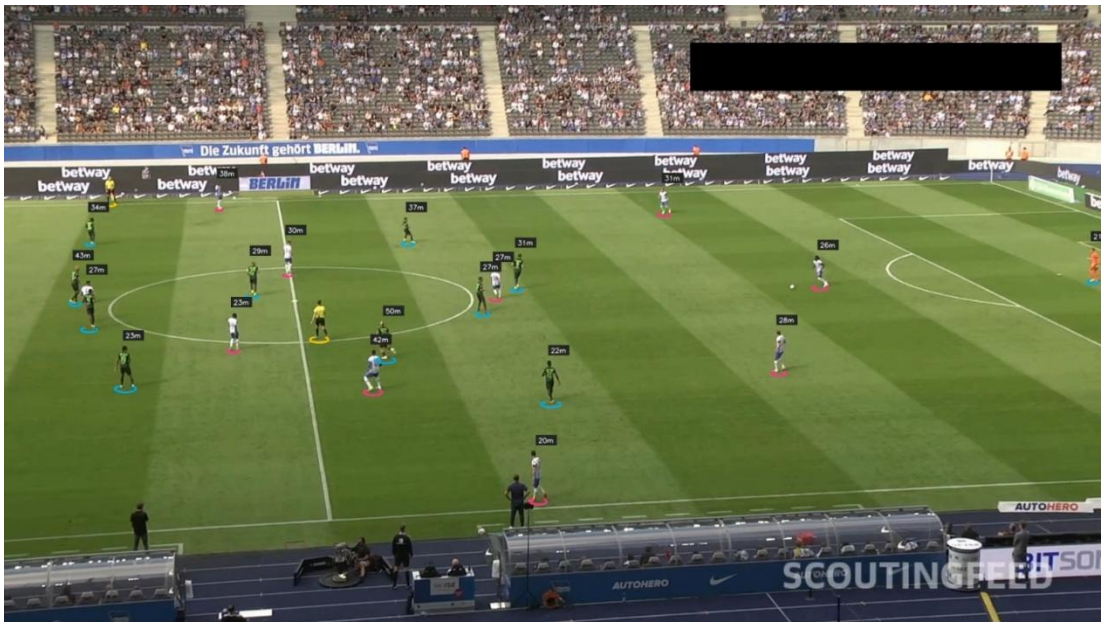


Figure 5.3: Broadcast View with distance

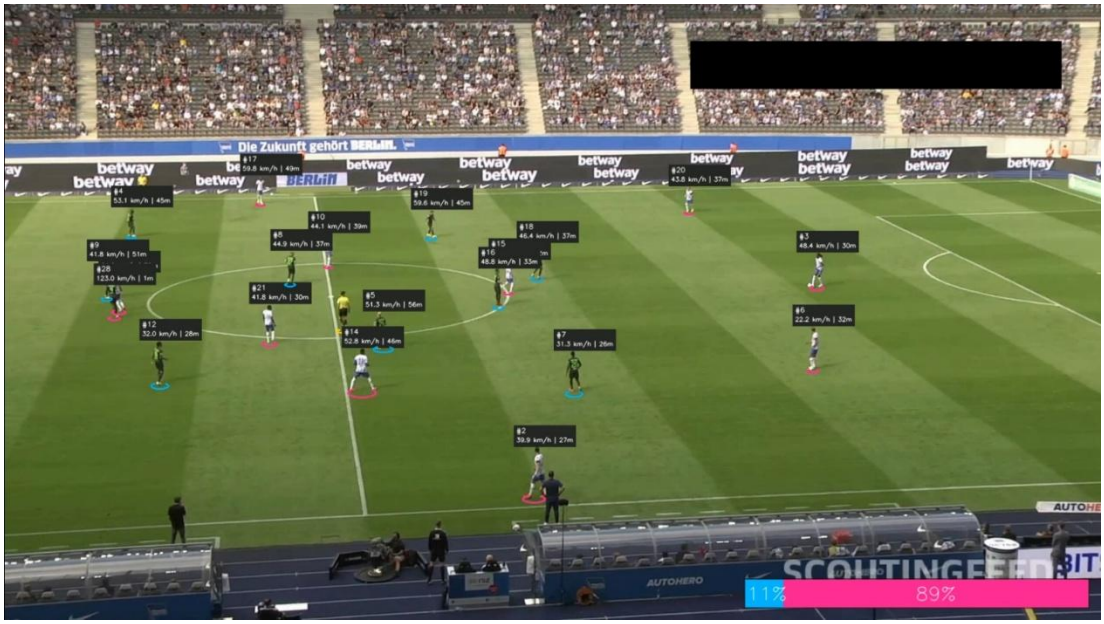


Figure 5.4: Broadcast View with combined metrics

5.2 Tactical Radar View Module

Tactical Radar View changes the view from an inclined broadcast view to a topographic view of space. It identifies the bottom-center location of players and uses the homography matrix to overlay them onto a scale model of a digital football pitch. This football pitch is subsequently divided into Voronoi regions, which indicate the territorial advantage based on player proximity.

Homographies from OpenCV software help remove perspective distortions. The Numpy meshgrids, together with Euclidean distance formulas, enable efficient creation of Voronoi regions. These regions are combined with the football pitch using image blending techniques, with the pitch and players drawn using Roboflow annotators.

This program works effectively in tactical planning, allowing coaches to analyze the compactness of the team, territorial advantage, and structural weaknesses of the team.

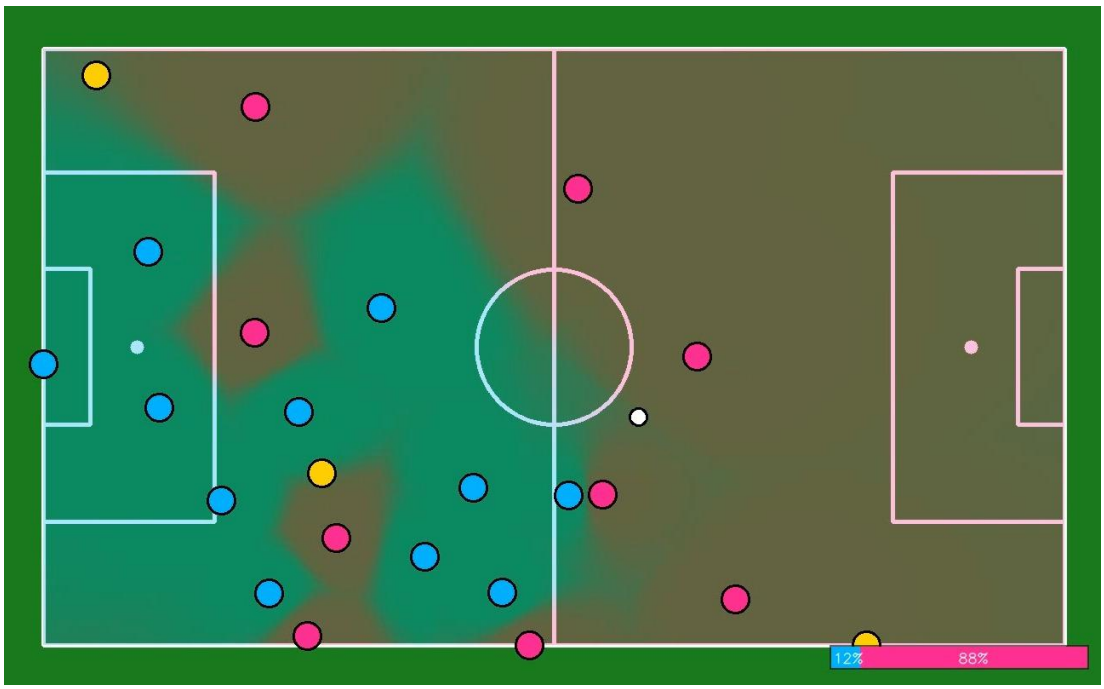


Figure 5.5: Radar View Voronoi diagram

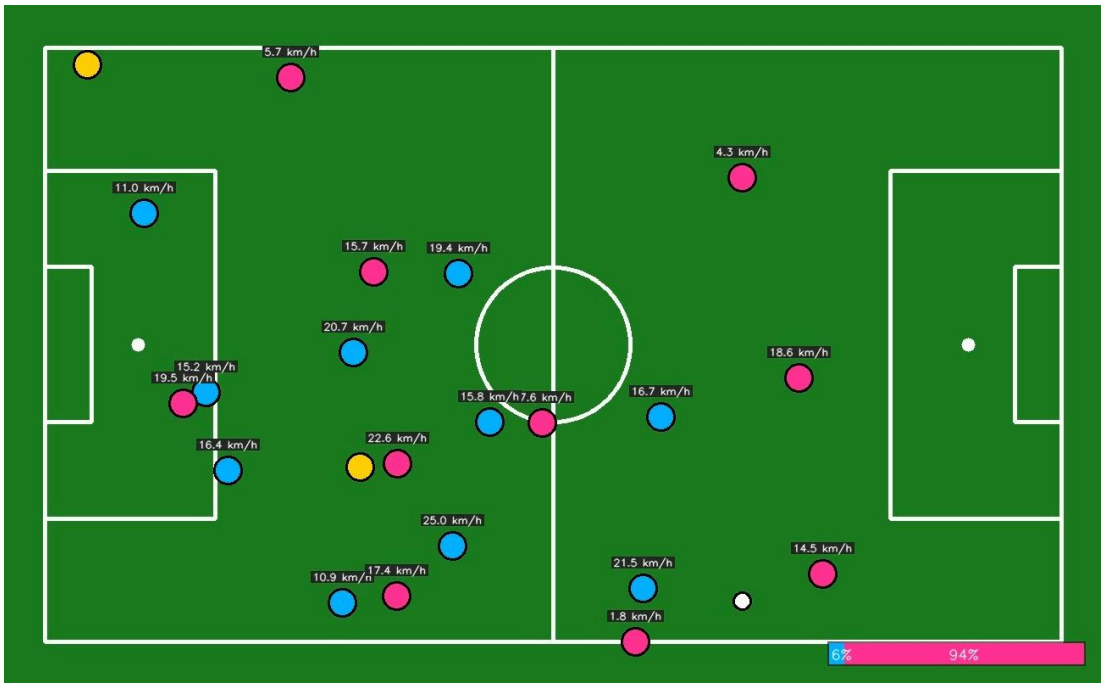


Figure 5.6: Radar View with speed

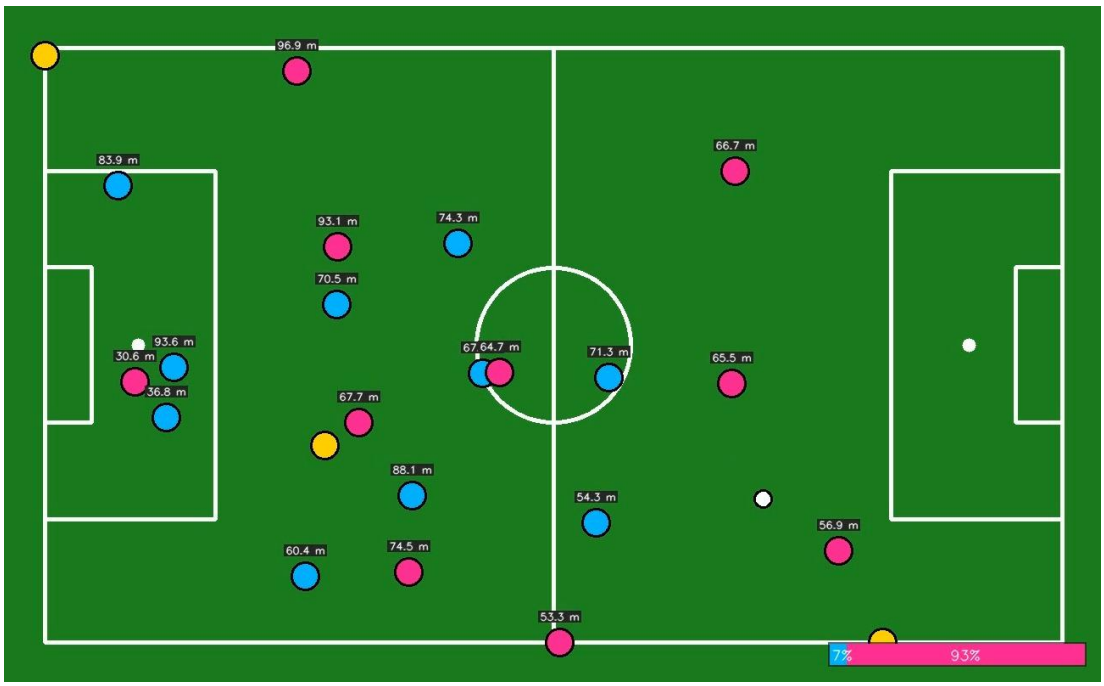


Figure 5.7: Radar View with distance

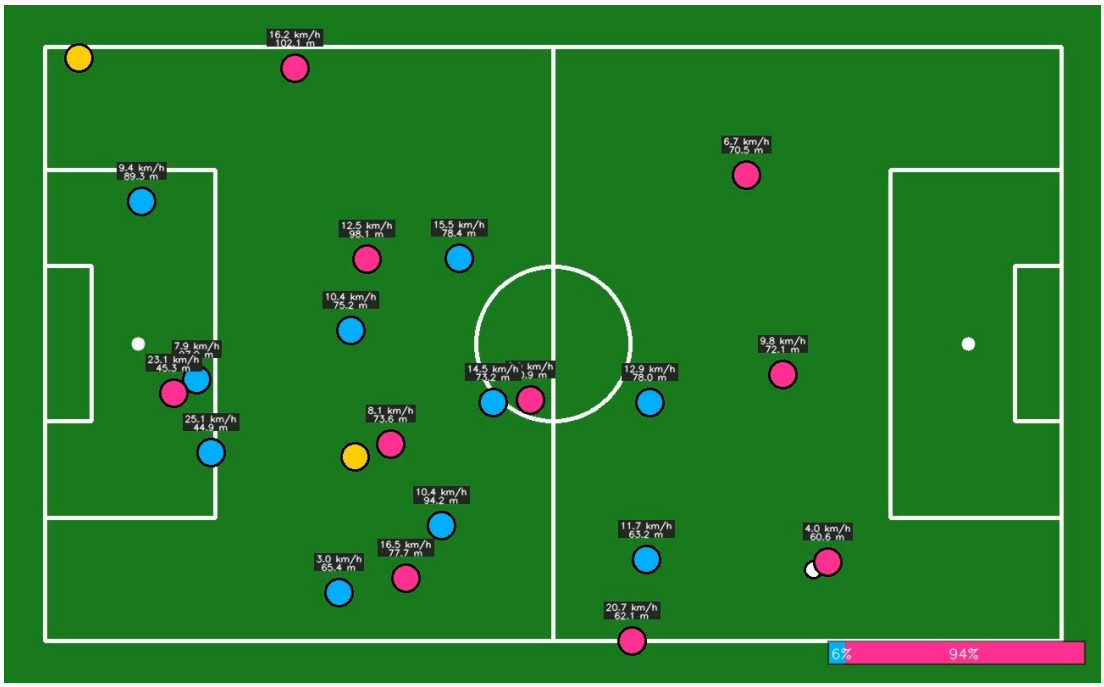


Figure 5.8: Radar View with speed and distance

5.3 Speed And Distance Tracking Module

In this module, we determine the speed of the player and the distance covered based on the homography-mapped coordinates. Speed is calculated by determining the positional difference between two frames and dividing by time.

Lucas Kanade optical flow deals with camera motion. Filters such as moving averages help to stabilize the velocity computations. Numpy library is used to calculate distance and convert the measurements to speed and total distance.

It provides performance statistics such as speed and distance covered, which would have otherwise been provided by a GPS tracking system.

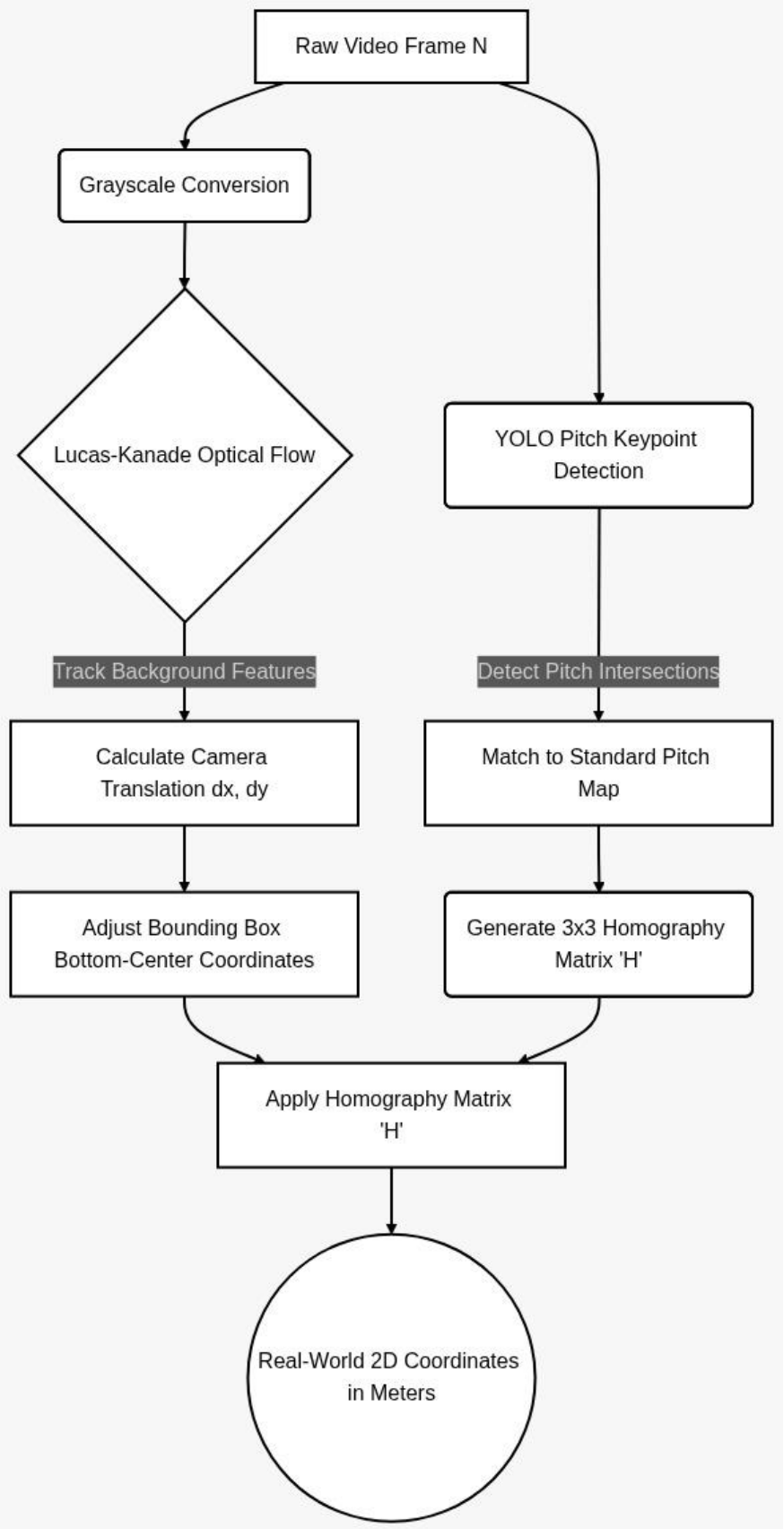


Figure 5.9: Mapping of 2-D pixel on a top-down pitch

5.4 Ball Possession Tracking Module

Control is determined by the possession module through the calculation of the distance between the ball and players. Whichever player has the shortest distance to the ball is declared as possessing the ball.

Fastest calculation of the nearest neighbor algorithm is facilitated by KD-Trees & Numpy while temporal buffering facilitates stability of possession. Graphical overlays indicate the possession percentage at any one time.

The possession module automates an important football metric which makes evaluation of strategy for teams easier.

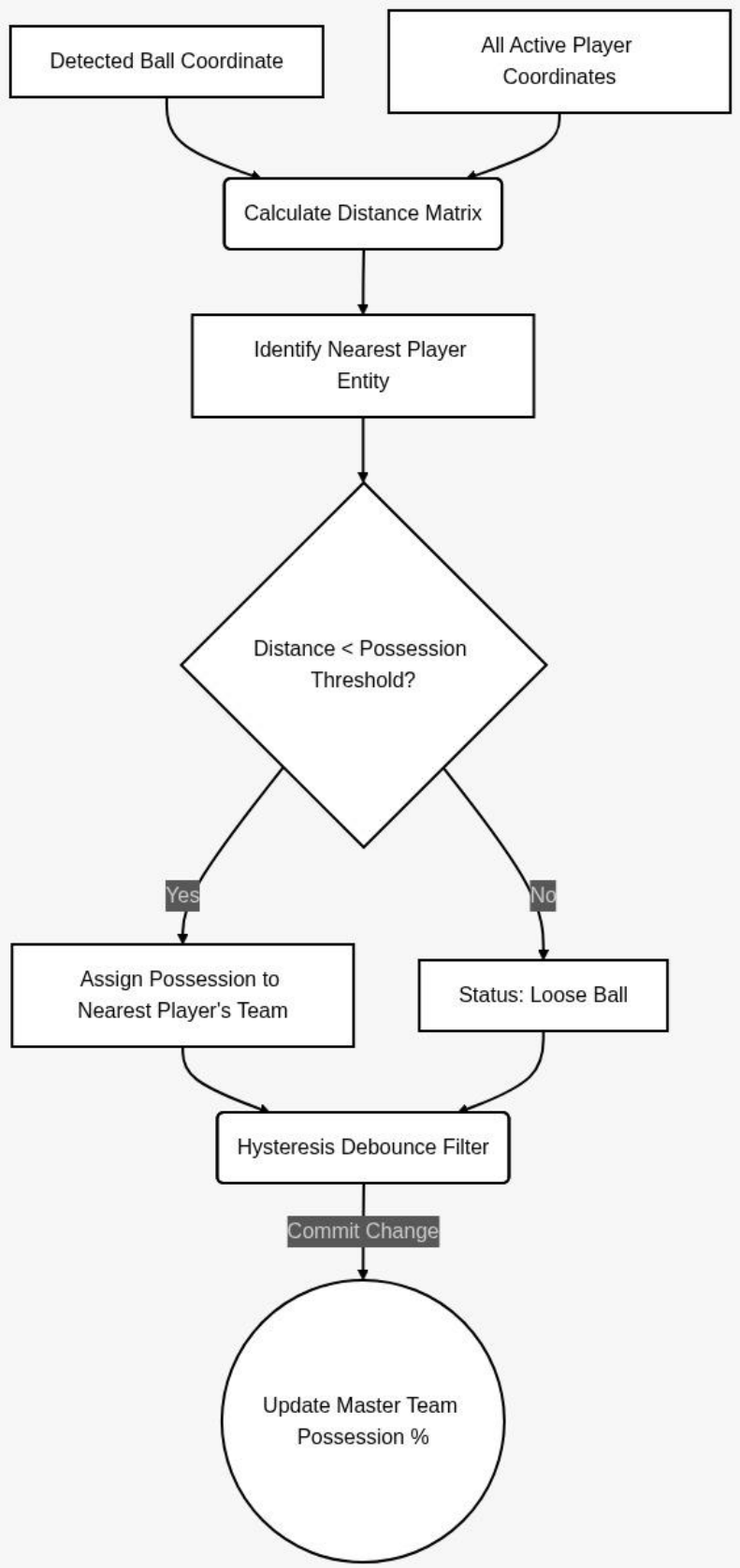


Figure 5.10: Ball Possession state machine

5.5 Combined Analytics Rendering Module

The final module merges results from all other modules into a single visualization. It merges tracking, speed, mapping, and possession information into a combination of visualization forms like a radar diagram with performance statistics.

OpenCV is employed to design expanded visual diagrams. Multithreading controls multiple streams of data. The object-oriented model of rendering handles the drawing task.

It offers a full analysis view for making decisions, which allows monitoring physical performance, positioning, and game situation at the same time.

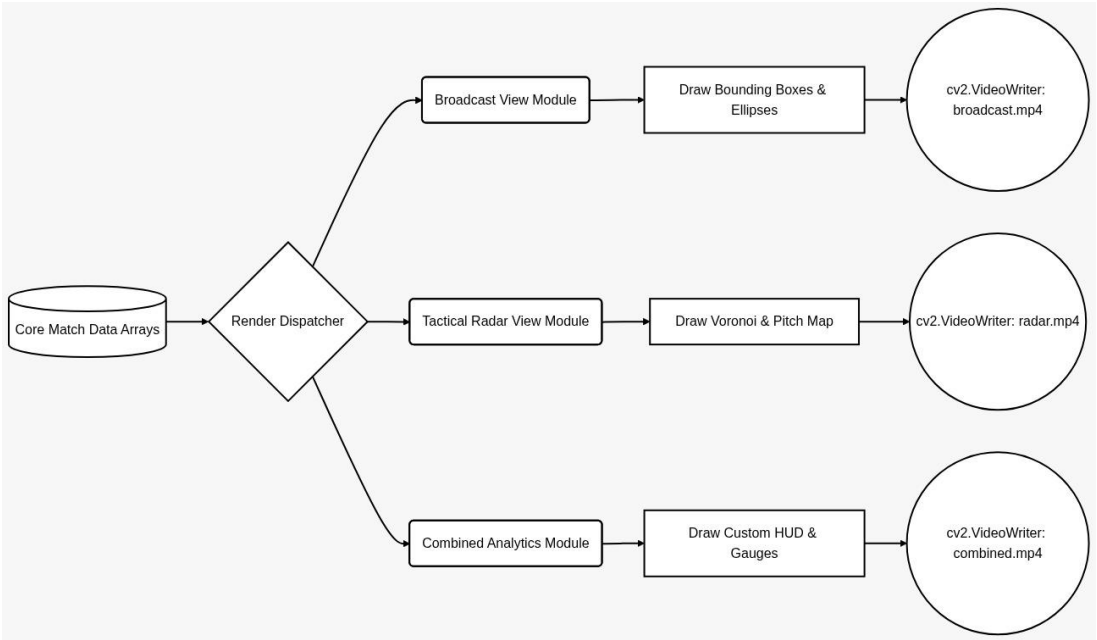


Figure 5.11: Multi-View Rendering Pipeline

CHAPTER 6

SYSTEM TESTING AND EVALUATION

6.1 Test Case: Object Detection Reliability and Occlusion Profiling

The core model relies on the YOLO26 [12] inference engine operating within specific tensor-core constraints.

Evaluation Methodology: On a selective dataset of 5 minutes of curated data, including 3 different demanding scenarios, we trained the pipeline: Extreme crowd packing inside 18-yard box, long-ball aerial sequences, and close-quarters dribbling overlapping referees.

Table 6.1: Object Detection Regression Profiles

Test ID	Execution Parameters & Scenarios	Target Key Performance Indicator	Observed Status/Final Grade
TC-OD-01	High-density player overlapping (corner-kick scenario containing >14 entities overlapping visually).	Recall Rate > 85%. Maintain minimum viable bounding boxes around player heads/shoulders regardless of leg occlusion.	**Pass.** Detection confidence dipped to roughly 35% logic margins, but ByteTrack successfully inherited those low confidence track maintaining IDs cleanly.
TC-OD-02	Referee classification with Dark kits	Class ID matches 2 persistently > 98% of frames	**Pass.** Contextual bounding boxes map the whistle with black shorts configuration for

			referee to separate him from team players
TC-OD-03	Football tracking during powerful clearance	Persistent bounding box mapping with maximum gap loss < 4 frames	**Conditional Pass.** Extreme motion blur washes away the ball but linear array interpolation natively patches the misses 2 frames.

6.2 Trajectory Stability Matrix

Multiple Object Tracking Accuracy (MOTA) and Identity Switching (IDSW) metric evaluations heavily dictate data collection validity.

Evaluation Methodology: Finding a mapping of particular players that incurs integer outputs that certify that they do not fracture. Should Player X move off-screen, then on returning to do so, he/she must be Player X, otherwise, one must cycle accordingly, without corrupting the data of Player Y.

Table 6.2: Trajectory Stability Matrix

Test ID	Execution Parameters & Scenarios	Target Key Performance Indicator	Observed Status/Final Grade
TC-TRK-01	Player A runs cleanly behind Player B	IDs maintain specific bounding post intersection logic	**Pass.** Intersection over union matrices bound to the Kalman velocity, prediction

			smoothly carry the inertia vector separating them correctly.
TC-TRK-02	Player is fully obscured behind goal nets for > 2 Seconds	Track ID terminated. Cache memory cleared.	**Pass.** Ghost tracks cleared correctly.
TC-TRK-03	Goalkeeper unique association bounds.	Goalkeeper remains classified separated from generic defense blocks.	**Pass.** Mathematically proximity bounds check successfully binds GK constraints permanently.

6.3 Transforming Clustering Resilience

Validating that the SigLIP feature extraction completely bypasses heuristic color limitations preventing fundamental clustering breakdown under lighting scenarios.

Evaluation Methodology: A video clip with a bold, severe diagonal roof of the stadiums shadowing the pitch was used. A white-kit team was used dynamically on both the completely lit grass as well as the heavily shadowed grass making HSV clustering tools fail.

Table 6.3: Transforming Clustering Resilience Matrix

Test ID	Execution Parameters & Scenarios	Target Key Performance Indicator	Observed Status/Final Grade

TC-CLS-01	Illumination variation classification	Clustering Integer maps remain uniform regardless of player position	**Pass.** UMAP clusters mapped distinctly to 2 domains perfectly.
TC-CLS-02	Mitigation of skin tone heuristic bias.	System does not prioritise limb color rather relationships mapped based on team colors.	**Pass.**

6.4 Planar Translation Mathematics

Testing of the geometric arrays assures that reporting on speed is consistent with massively real-world physics and does not give unrealistic human physical values (e.g. telling a player running at 60 km/h).

Table 6.4: Planar Translation Mathematics Matrix

Test ID	Execution Parameters & Scenarios	Target Key Performance Indicator	Observed Status/Final Grade
TC-MATH-01	Zero-velocity static stability checking.	Stationary player while camera pans violently left. Tracker outputs max Speed < 1 km/h.	**Pass.** Dense optical flow effectively reverses the global displacement canceling the matrix shifts completely.

TC-MATH-02	Sprinting threshold verification.	Observing a professional winger sprint down the touchlines evaluating raw velocity arrays.	**Pass.** Data outputs peaked around 33.2 km/h, well within standard human physiological top-end parameters.

CHAPTER 7

CONCLUSION

7.1 Conclusion Statement

Having provided a complete test run of an elite-level, mathematically rigorous football analytical pipeline that is no longer a secretive billion-dollar business infrastructure, this project shows that such pipelines can now be publicly accessed free of charge. Through the synthesis of state-of-the-art YOLOv26 [15] localization , tenacious ByteTrack linkage parameter and zero-shot SigLIP visual transformers coupled with elaborate geometric planar homography protocols, FUTVIZ has been able to build an uncompromised automated framework.

The application attains its main goals in a uniform manner: it can assess athletic results without errors, multi-view visualizations that map standard broadcasts, and overcome systemic biases in visual categories that older computer vision applications are fatally prone to exhibit. My platform creates a direct connection to a democratized access to professional tactical information in academies worldwide.

7.2 Core Architectural Optimizations (Future Work)

Although the localized framework is profoundly stable, there are a number of architectural extensions that map out the journey towards migrating into a SaaS (Software as a Service) enterprise paradigm:

7.2.1 Edge Deployment and Hardware Acceleration

Currently, executing the heavy PyTorch SigLIP pipeline introduces heavy overhead. By compiling the model architectures natively down via NVIDIA TensorRT mappings or converting the models to robust ONNX topologies, execution latency could easily be halved, bridging the gap towards genuine live-feed ingestion protocols.

7.2.2 Large Language Model (LLM) Integration

With arrays mapping thousands of coordinate sequences determining possession algorithms and crossing formations, routing this compiled CSV data through an LLM sequence generator could autonomously produce textual tactical summaries (e.g. The

right-wing overlaps generated 60% of box entries, primarily bypassing Team B's high defensive array block.") replacing manual coaching interpretations.

7.3 Advanced Tactical Output Horizons

Currently, the tactical radar representation outputs 2D coordinate spaces mapping general tactical formations. By executing multi-camera calibration topologies (leveraging synchronization arrays from 3 scattered 4K stadium cameras natively), true 3D spatial mapping can govern highly advanced operations such as generating autonomous VAR-style offside 3D line projections natively upon the video feeds.

7.4 Systemic Authentication and Data Protocol Security

Should the system transition to a web-application environment via FastAPI or Flask cloud dockers, stringent cybersecurity guidelines controlling physical metric tracking parameters must be implemented. Establishing granular Role Based Access Control (RBAC) ensuring player physiological data is securely siloed according strictly to international GDPR implementations surrounding biometric tracking standards forms the necessary foundation for future deployments.

REFERENCES

[1] E. M. G. M. Hoseinzadeh Z, “Analysis of research structure and scientific trends in artificial intelligence, machine learning, and deep learning in football: a bibliometric approach,” International Journal of Intelligent Computing and Cybernetics, 2026.

[2] M. S. K. I. F. S. A. N. V. V. S. J. O. &. A. S. S. Vishwa, “ A Study on Object Detection Using YOLO Algorithm and It Is Real-Time Application in Sports Field-Detection,” In Proceedings of International Conference on Computing Systems and Intelligent Applications, 2026, January.

[3] N. K. S. T. K. G. M. &. A. A. Batbaatar, “ANTHROPOMETRIC BODY ZONE-BASED SOCCER PASS DETECTION AND TEAM CLASSIFICATION WITH INTEGRATED COMPUTER VISION MODELS.,” 2026.

[4] E. M. Hoseinzadeh Z, “Hoseinzadeh Z, E. M. (2026). Analysis of research structure and scientific trends in artificial intelligence, machine learning, and deep learning in football: a bibliometric approach. International Journal of Intelligent Computing and Cybernetics.,” 2026.

[5] M. A. K. B. P. &. B. M. Klemmer, “Automating assist identification in football (soccer): a machine learning approach using event and tracking data.”.

[6] (. SkillCorner, “SkillCorner. (2026). Physical Performance Tracking: Technical Whitepaper. Retrieved from https://www.skillcorner.com/”.

[7] 2. Roboflow, “Roboflow (2026) Computer Vision in Sports: A Comprehensive Guide. Retrieved from https://blog.roboflow.com/computer-vision-sports-guide/”.

[8] Face, H. (2024) Transformers Library, “Face, H. (2024). Transformers Library - Vision Models Documentation. Retrieved from https://huggingface.co/docs/transformers/index,” 2024.

[9] O. (2024), “OpenCV. (2024). OpenCV 4.x Documentation - Video Analysis and Object Tracking.”.

[10] PyTorch Vision (2024), “PyTorch. (2024). PyTorch Vision (torchvision) - Models and Utilities. Retrieved from https://pytorch.org/vision/stable/index.html”.

[11] T. L. Y. &. W. J. Zhang, “Ensemble methods for human action recognition in video sequences. Computer Vision and Image Understanding, 208, 103220. https://doi.org/10.1016/j.cviu.2021.103220,” 2021.

[12] R. &. K. M. (. Y.-2. I. Y. w. Y. f. R.-T. O.-V. I. S. a. p. a. Sapkota, “YOLOE-26: Integrating YOLO26 with YOLOE for Real-Time Open-Vocabulary Instance Segmentation,” 2026.

[13] P. S. (. Hidayatullah, “Hidayatullah, P. S. (2025). YOLOv8 to YOLO11: A comprehensive architecture in-depth comparative review. .”.

[14] R. C. R. H. S. A. &. K. M. (. Sapkota, “Sapkota, R., Cheppally, R. H., Sharda, A., & Karkee, M. (2025). YOLO26: key architectural enhancements and performance benchmarking for real-time object detection. arXiv preprint arXiv:2509.25164.,” 2025.

[15] R. &. K. M. Sapkota, “ YOLOE-26: Integrating YOLO26 with YOLOE for Real-Time Open-Vocabulary Instance Segmentation,” 2026.

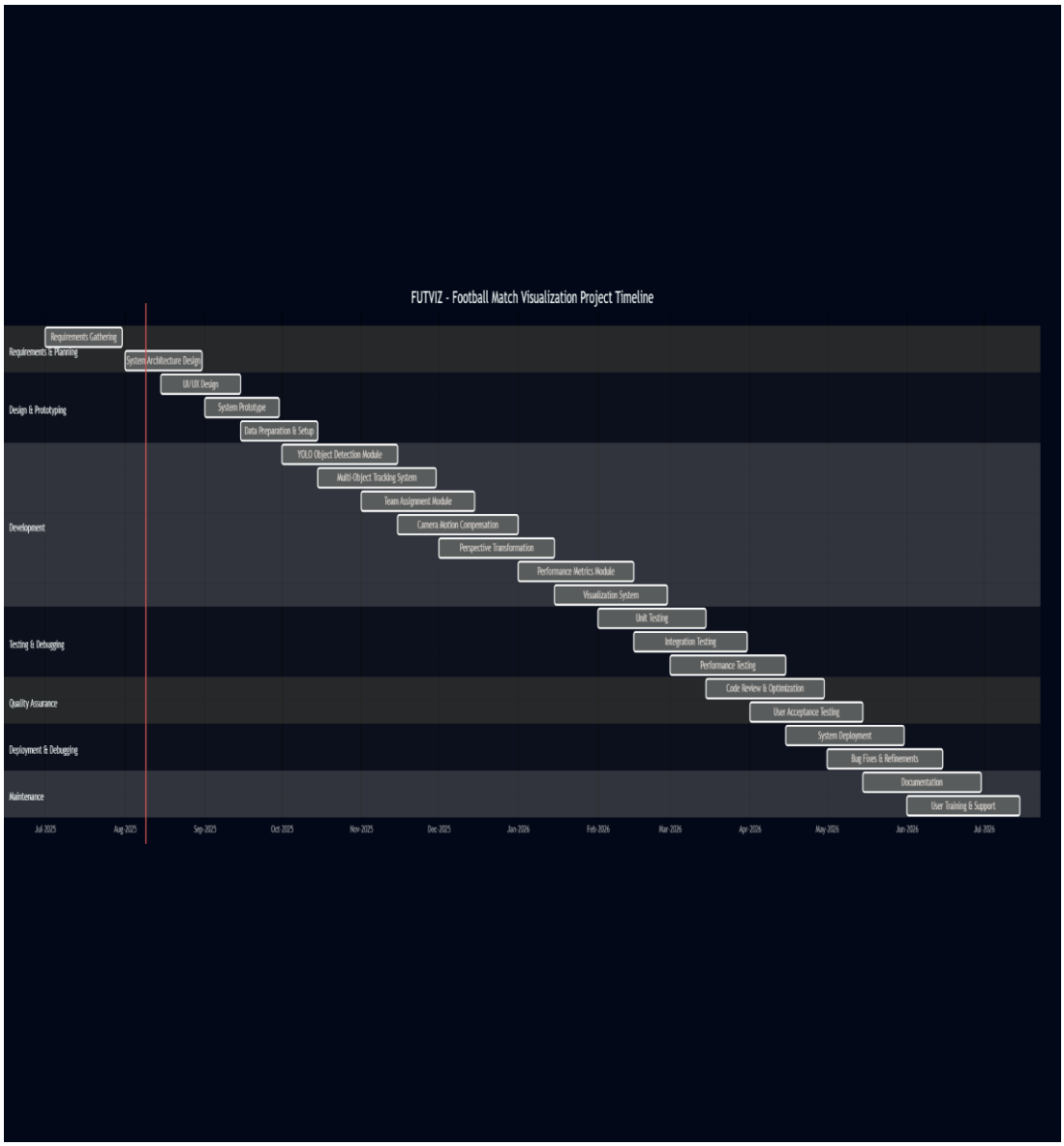
[16] S. D. a. E. 2. E. A. H. ,. B. M. 2. M. Barkat, “ M. Barkat, Signal Detection and Estimation, 2nd Edition, Artech House , Boston, MA., 2005”.

[17] O. (. O. 4. D. -. V. A. a. O. Tracking, “OpenCV. (2024). OpenCV 4.x Documentation - Video Analysis and Object Tracking”.

APPENDICES

APPENDIX A: Graphs

The Gantt chart shows the complete development timeline of FUTVIZ FYP Project from October 2025 to May 2026 starting from Initiation to final Deployment. It shows keysteps like looking into a problem or issue, designing the system, preparing data set, developing the AI model, implementing backend O&F, testing and debugging and finally deployment. The Gantt chart reflects a systematic development approach taken in transforming FUTVIZ into a football match visualization and player performance tracker using AI platform from an initial idea.



APPENDIX B: Computer Programme Listing (CODE)

Centralised pipeline constants (`config.py`)

```python
# Excerpt: Backend/config.py (detection, tracking, team vote window, crop stride)

DETECTION_CONFIDENCE       = 0.2
NMS_THRESHOLD              = 0.5
SMALL_AREA_PX2             = 1500
SMALL_AREA_CONF_RELAX_FACTOR = 0.5

TRACK_ACTIVATION_THRESHOLD = 0.20
LOST_TRACK_BUFFER          = 60
MINIMUM_MATCHING_THRESHOLD = 0.8
MINIMUM_CONSECUTIVE_FRAMES = 2

CLASS_VOTE_WINDOW          = 15   # referee class temporal window
MAX_REFEREE_COUNT          = 3

TEAM_VOTE_WINDOW           = 11
TEAM_VOTE_MIN_FOR_LOCK     = 3
```

```
CROP_STRIDE              = 20


# Homography (rolling mean + display EMA + spike rejection)
HOMOGRAPHY_WINDOW_SIZE     = 15
HOMOGRAPHY_DISPLAY_EMA_ALPHA = 0.12
KEYPOINT_CONFIDENCE_THRESH  = 0.5
KEYPOINT_CONFIDENCE_FALLBACK = 0.35
MIN_KEYPOINTS_FOR_HOMOGRAPHY = 4
MAX_REPROJECTION_ERROR_CM   = 800.0
HOMOGRAPHY_MAX_H_FROB_DELTA   = 42.0
HOMOGRAPHY_SPIKE_MIN_HISTORY  = 5


# IDMemory + possession + speed gating
ID_MEMORY_MAX_DISTANCE_PX   = 80
ID_MEMORY_MAX_FRAMES_MISSING = 60
SPEED_SMOOTHING_WINDOW     = 5
MAX_PLAUSIBLE_DELTA_M      = 5.0
MAX_BALL_PLAYER_DISTANCE_M  = 2.0
POSSESSION_GRACE_FRAMES    = 10


OUTPUT_DIR = "Backend/Output_Videos"
```

 # ... OUTPUT_PATHS: view_1 ... view_8 (see Listing J.1)

```

***Model paths (align with §4.3 / dataset narrative).***

```python

*# Excerpt: Backend/config.py — local weights*

YOLO_MODEL_PATH = "Backend/models/best_yolo26.pt"

PITCH_KP_MODEL_PATH = "Backend/models/football_field_detection_model.pt"

SIGLIP_MODEL_PATH = "Backend/models/siglip"

DEVICE = "cuda" *# or "cpu"*

```

Strided crop collection for `TeamAssigner.fit`

strided frames, stricter crop confidence, NMS, player-only crops (matches §4.2.1).

```python

```
Excerpt: Backend/main.py — Phase 2 crop collection + Phase 3 TeamAssigner.fit

log("\n[Phase 2] Collecting crops (strided)...")
torch.cuda.empty_cache()

crops = []
# Crop collection runs at the *original* (higher) confidence so the team
# classifier is trained on clean, high-quality samples.
CROP_COLLECTION_CONF = max(DETECTION_CONFIDENCE, 0.3)
for frame in tqdm(
    sv.get_video_frames_generator(VIDEO_PATH, stride=CROP_STRIDE),
    desc="Crop collection",
):
    try:
        result = yolo_model.predict(
            frame, conf=CROP_COLLECTION_CONF, device=0, verbose=False
)[0]
        dets = sv.Detections.from_ultralytics(result)
        dets = dets.with_nms(threshold=NMS_THRESHOLD, class_agnostic=True)
        players = dets[dets.class_id == PLAYER_ID]
        crops += [sv.crop_image(frame, xyxy) for xyxy in players.xyxy]
```

```
    except Exception as exc:
        log(f"  [WARN] crop error: {exc}")

if len(crops) == 0:
    log("[FATAL] Zero crops — cannot train TeamAssigner. Aborting.")
    sys.exit(1)

log("\n[Phase 3] Training TeamAssigner...")
team_assigner = TeamAssigner(device=DEVICE)
team_assigner.fit(crops)
del crops
torch.cuda.empty_cache()
```

`TeamAssigner.fit`: batched SigLIP embeddings, UMAP fit, K-Means fit, then `_compute_team_colors` (§4.5.1–4.5.4).

```python
# Excerpt: Backend/team_assigner/team_assigner.py — fit()

def fit(self, crops):
    if not crops:
        print("No crops provided for training.")
```

```
        return

    pil_crops = [sv.cv2_to_pillow(crop) for crop in crops]
    data = []
    batch_size = 32

    with torch.no_grad():
        for i in tqdm(range(0, len(pil_crops), batch_size), desc='Embedding extraction'):
            batch = pil_crops[i:i+batch_size]
            inputs = self.processor(images=batch, return_tensors="pt").to(self.device)
            outputs = self.model(**inputs)
            embeddings = torch.mean(outputs.last_hidden_state, dim=1).cpu().numpy()
            data.append(embeddings)

    self.embeddings = np.concatenate(data)
    print("Fitting UMAP and KMeans...")
    self.projections = self.reducer.fit_transform(self.embeddings)
    self.cluster_model.fit(self.projections)
    self._compute_team_colors(crops)
```

Centre-patch per crop; median across players per cluster (robust kit colour).

```python
# Excerpt: Backend/team_assigner/team_assigner.py — _compute_team_colors()


labels = self.cluster_model.labels_
team_crops = {0: [], 1: []}
for i, label in enumerate(labels):
    team_crops[label].append(crops[i])


for team_id, crop_list in team_crops.items():
    patch_colors = []
    for img in crop_list:
        h, w, _ = img.shape
        start_y, end_y = int(h * 0.25), int(h * 0.75)
        start_x, end_x = int(w * 0.25), int(w * 0.75)
        center_patch = img[start_y:end_y, start_x:end_x]
        if center_patch.size > 0:
            avg_color = np.mean(center_patch, axis=(0, 1))
            patch_colors.append(avg_color)
    if patch_colors:
        median_color = np.median(np.array(patch_colors), axis=0)
```

```
        self.team_colors[team_id] = median_color
```

Inference-time `predict`: `transform` (fitted UMAP) + `predict` (fitted K-Means); optional confidence from centroid distances (§4.2.3 / ambiguity handling).

```python
# Excerpt: Backend/team_assigner/team_assigner.py — predict()
# (After the same batched SigLIP loop as in fit(), storing embeddings in `data`.)

embeddings = np.concatenate(data)
projections = self.reducer.transform(embeddings)
labels = self.cluster_model.predict(projections)

# Optional confidence (Task 6): distance to own vs other centroid
centroids = self.cluster_model.cluster_centers_
diffs = projections[:, None, :] - centroids[None, :, :]
dists = np.linalg.norm(diffs, axis=2)
d_assigned = dists[np.arange(len(labels)), labels]
d_other = dists[np.arange(len(labels)), 1 - labels]
denom = np.where((d_assigned + d_other) < 1e-9, 1e-9, d_assigned + d_other)
confidence = np.clip((d_other - d_assigned) / denom, 0.0, 1.0)
```

ByteTrack, stable IDs, team + referee voting. ***Class IDs from YOLO metadata (portable across retrained heads).***

```python
# Excerpt: Backend/main.py — after loading yolo_model

_names_inv = {v: k for k, v in yolo_model.names.items()}
BALL_ID    = _names_inv[CLASS_BALL]
GK_ID      = _names_inv[CLASS_GOALKEEPER]
PLAYER_ID  = _names_inv[CLASS_PLAYER]
REFEREE_ID = _names_inv[CLASS_REFEREE]
```

Detection + small-bbox confidence relaxation + ByteTrack (§4.3–4.4).*

```python
# Excerpt: Backend/main.py — per-frame detection → track

result = yolo_model.predict(frame, conf=DETECTION_CONFIDENCE,
                            device=0, verbose=False)[0]
dets = sv.Detections.from_ultralytics(result)
```

```
non_ball = dets[dets.class_id != BALL_ID]

if len(non_ball) > 0 and non_ball.confidence is not None:

    bw = non_ball.xyxy[:, 2] - non_ball.xyxy[:, 0]

    bh = non_ball.xyxy[:, 3] - non_ball.xyxy[:, 1]

    areas = bw * bh

    small_mask = areas < SMALL_AREA_PX2

                relaxed_conf = DETECTION_CONFIDENCE * SMALL_AREA_CONF_RELAX_FACTOR

    keep = (

        (non_ball.confidence >= DETECTION_CONFIDENCE)

        | (small_mask & (non_ball.confidence >= relaxed_conf))

    )

    non_ball = non_ball[keep]

non_ball = non_ball.with_nms(threshold=NMS_THRESHOLD, class_agnostic=True)


non_ball = tracker.update_with_detections(non_ball)
```

`IDMemory` + `RefereeVoter` on **stable** ids; homography update; entity split; referee display cap (§4.2.3, §4.4.2).

```python
id_memory.begin_frame()
stable_ids = []
if non_ball.tracker_id is not None and len(non_ball) > 0:
    anchors = non_ball.get_anchors_coordinates(sv.Position.BOTTOM_CENTER)
    for i in range(len(non_ball)):
        tid = int(non_ball.tracker_id[i])
        sid = id_memory.update(tid, (anchors[i, 0], anchors[i, 1]))
        stable_ids.append(sid)
stable_ids = np.array(stable_ids, dtype=int)


if non_ball.tracker_id is not None and len(non_ball) > 0:
    for i in range(len(non_ball)):
        sid = int(stable_ids[i])
        cls = int(non_ball.class_id[i])
        non_ball.class_id[i] = referee_voter.vote(
            tracker_id=sid,
            observed_class_id=cls,
            referee_class_id=REFEREE_ID,
        )
```

```
kps = pitch_detector.detect(frame)

transformer = homography.update(kps)


player_mask = non_ball.class_id == PLAYER_ID

gk_mask     = non_ball.class_id == GK_ID

ref_mask    = non_ball.class_id == REFEREE_ID

players_det   = non_ball[player_mask]

gk_det        = non_ball[gk_mask]

referees_det  = non_ball[ref_mask]

players_sids  = stable_ids[player_mask]

gk_sids       = stable_ids[gk_mask]

referees_sids = stable_ids[ref_mask]


if len(referees_det) > MAX_REFEREE_COUNT:
    if referees_det.confidence is not None:
       top_idx = np.argsort(referees_det.confidence)[::-1][:MAX_REFEREE_COUNT]
    else:
        top_idx = np.arange(MAX_REFEREE_COUNT)
    referees_det = referees_det[top_idx]
    referees_sids = referees_sids[top_idx]
```

Per-frame team labels + temporal `TeamVoter`; GK raw teams then smoothed (§4.2.3, §4.5, FR4).

```python
player_team_ids = np.zeros(len(players_det), dtype=int)
if len(players_det) > 0:
    player_crops = [sv.crop_image(frame, xyxy) for xyxy in players_det.xyxy]
    raw_player_teams = team_assigner.predict(player_crops)
    for i in range(len(players_det)):
        sid = int(players_sids[i])
        player_team_ids[i] = team_voter.vote(sid, int(raw_player_teams[i]))

gk_team_ids = np.zeros(len(gk_det), dtype=int)
if len(gk_det) > 0 and len(players_det) > 0:
    _tmp_players = copy.deepcopy(players_det)
    _tmp_players.class_id = player_team_ids.astype(int)
    raw_gk_teams = resolve_goalkeepers_team_id(_tmp_players, gk_det)
    for i, raw_team in enumerate(raw_gk_teams):
        sid = int(gk_sids[i])
        gk_team_ids[i] = team_voter.vote(sid, int(raw_team))
```

`RefereeVoter`: rolling-window majority + permanent lock per stable id.

```python
# Excerpt: Backend/team_assigner/voters.py — RefereeVoter


class RefereeVoter:
    def vote(self, tracker_id: int, observed_class_id: int, referee_class_id: int) -> int:
        votes = self._votes[tracker_id]
        votes.append(observed_class_id)
        if tracker_id in self._locked:
            return referee_class_id
        majority = Counter(votes).most_common(1)[0][0]
        if majority == referee_class_id:
            self._locked.add(tracker_id)
            return referee_class_id
        return majority
```

`TeamVoter`: first `min_votes_for_lock` frames pass through raw; then rolling majority (window = `TEAM_VOTE_WINDOW`).

```python
# Excerpt: Backend/team_assigner/voters.py — TeamVoter
```

```
class TeamVoter:
    def vote(self, tracker_id: int, observed_team_id: int) -> int:
        votes = self._votes[tracker_id]
        votes.append(int(observed_team_id))
        if len(votes) < self._min_lock:
            return int(observed_team_id)
        return Counter(votes).most_common(1)[0][0]
```

End-of-frame `IDMemory` tick (increments `frames_missing`, prunes stale stable ids).

```python
# Excerpt: Backend/main.py — after all frame_data built, before rendering

id_memory.end_frame()
```

Pitch keypoints and homography (planar mapping): *`PitchDetector.detect`: YOLO keypoint inference, best detection selection, origin-guard (§4.6.2).*

```python
# Excerpt: Backend/pitch_detection/pitch_detector.py

results = self.model.predict(frame, conf=0.3, device=self.device, verbose=False)
```

```
if not results or results[0].keypoints is None:
    return {}
result = results[0]
if result.keypoints.xy is None or len(result.keypoints.xy) == 0:
    return {}


if result.boxes is not None and len(result.boxes.conf) > 0:
    best_idx = int(result.boxes.conf.argmax())
else:
    best_idx = 0


xy   = result.keypoints.xy[best_idx]
conf = result.keypoints.conf[best_idx] if result.keypoints.conf is not None else None


keypoints_dict = {}
for kp_idx in range(len(xy)):
    x_val, y_val = float(xy[kp_idx][0]), float(xy[kp_idx][1])
    c_val = float(conf[kp_idx]) if conf is not None else 1.0
    if x_val == 0.0 and y_val == 0.0:
        continue
    keypoints_dict[kp_idx] = (x_val, y_val, c_val)
```

return keypoints_dict

```

***`HomographyTransformer.update`: confidence pairs, RANSAC `ViewTransformer`, median reprojection gate, rolling mean, display EMA, Frobenius spike rejection (§4.6.1, §4.6.3).***

```python

# Excerpt: Backend/pitch_detection/homography.py — core of update()

*def* **_pairs_for_thresh**(th: *float*) -> *tuple*[np.ndarray, np.ndarray]:

    fp, pp = [], []

    *for* idx, (x, y, conf) *in* keypoints.items():

        *if* idx >= len(self._pitch_vertices):

            *continue*

        *if* conf < th:

            *continue*

        fp.append([x, y])

        pp.append(self._pitch_vertices[idx].tolist())

    *if* len(fp) < self._min_pairs:

        *return* np.empty((0, 2), np.float32), np.empty((0, 2), np.float32)

    *return* np.asarray(fp, dtype=np.float32), np.asarray(pp, dtype=np.float32)

src, dst = _pairs_for_thresh(self._conf_thresh)

*if* len(src) < self._min_pairs and self._conf_fallback < self._conf_thresh:

    src2, dst2 = _pairs_for_thresh(self._conf_fallback)

    *if* len(src2) >= self._min_pairs:

        src, dst = src2, dst2

raw_transformer = ViewTransformer(source=src, target=dst)

H_raw = raw_transformer.m

*# ... determinant gate, median reprojection vs MAX_REPROJECTION_ERROR_CM ...*

self._M.append(H_raw)

H_roll = np.mean(np.stack(self._M, axis=0), axis=0)

a = self._display_ema_alpha

H_out = H_roll *if* self._H_display is None *else* a * H_roll + (1.0 - a) * self._H_display

*# Spike guard: if ‖H_out − H_prev‖_F too large, pop last H and keep previous transformer*

*# ... then self._transformer.m = H_out ...*

```

```

Goalkeeper team resolution (spatial prior): ***Early exit if only one team visible; else centroid + side prior (§4.2.3 goalkeeper paragraph).***

```python
# Excerpt: Backend/team_assigner/utils.py — resolve_goalkeepers_team_id (guards + loop)


if len(players) == 0 or len(goalkeepers) == 0:
    return np.array([])


mask_0 = players.class_id == 0
mask_1 = players.class_id == 1
if not np.any(mask_0) or not np.any(mask_1):
    return np.zeros(len(goalkeepers), dtype=int)


left_team = 0 if team_0_centroid[0] <= team_1_centroid[0] else 1
side_weight = 2.0
for goalkeeper_xy in goalkeepers_xy:
    scores = {}
    for team in (0, 1):
        centroid_dist = np.linalg.norm(goalkeeper_xy - centroids[team])
        if team == left_team:
            side_edge = np.percentile(team_x[team], 35)
```

```
            side_penalty = max(0.0, goalkeeper_xy[0] - side_edge)
        else:
            side_edge = np.percentile(team_x[team], 65)
            side_penalty = max(0.0, side_edge - goalkeeper_xy[0])
        scores[team] = centroid_dist + side_weight * side_penalty
    goalkeepers_team_id.append(0 if scores[0] <= scores[1] else 1)
```

Broadcast overlay (ellipses, IDs, ball, possession bar): ***Full `BroadcastRenderer`: referees (ellipse only), players (ellipse + `#id` + speed), GKs, ball triangle, possession bar (§5.1).***

```python
# Excerpt: Backend/renderers/broadcast_renderer.py

class BroadcastRenderer:
    def render(self, frame: np.ndarray, frame_data: dict) -> np.ndarray:
        out = frame.copy()

        for tid, rdata in frame_data.get("referees", {}).items():
            draw_ellipse_marker(out, rdata["bbox"], get_entity_color("referee"))
```

```
for tid, pdata in frame_data.get("players", {}).items():
    color = get_entity_color("player", pdata.get("team_id", 0))
    draw_ellipse_marker(out, pdata["bbox"], color)
    bbox = pdata["bbox"]
    cx = (bbox[0] + bbox[2]) / 2
    draw_outlined_text(out, f"#{tid}", (cx - 10, bbox[3] + 15),
                       font_scale=0.35, color=color)
    speed = pdata.get("speed_kmh", 0.0)
    if speed > 0.5:
        draw_outlined_text(
            out, f"{speed:.1f} km/h",
            (cx - 20, bbox[1] - 8),
            font_scale=0.35,
        )

for tid, gdata in frame_data.get("goalkeepers", {}).items():
    color = get_entity_color("goalkeeper", gdata.get("team_id", 0))
    draw_ellipse_marker(out, gdata["bbox"], color)
    bbox = gdata["bbox"]
    cx = (bbox[0] + bbox[2]) / 2
```

```
            draw_outlined_text(out, f"#{tid}", (cx - 10, bbox[3] + 15),
                               font_scale=0.35, color=color)

        ball = frame_data.get("ball", {})
        if ball.get("bbox") is not None:
            draw_triangle_marker(out, ball["bbox"])

        poss = frame_data.get("possession", {})
        draw_possession_bar(
            out,
            poss.get("team_0_pct", 0.0),
            poss.get("team_1_pct", 0.0),
            bar_w=600,
            bar_h=48,
        )
        return out
```

Speed-only view (View 2) — neutral label background (§5.3).

```python
# Excerpt: Backend/renderers/speed_renderer.py — SpeedRenderer.render()

for category in ("players", "goalkeepers"):
```

```
    for tid, edata in frame_data.get(category, {}).items():
        bbox  = edata["bbox"]
        speed = edata.get("speed_kmh", 0.0)
        color = get_entity_color("player", edata.get("team_id", 0))
        draw_ellipse_marker(out, bbox, color)
        cx = int((bbox[0] + bbox[2]) / 2)
        y  = int(bbox[1]) - 12
        text = f"{speed:.1f} km/h"
        (tw, th), _ = cv2.getTextSize(text, cv2.FONT_HERSHEY_SIMPLEX, 0.38, 1)
        cv2.rectangle(out, (cx - 4, y - th - 4), (cx + tw + 4, y + 4), _LABEL_BG, -1)
            cv2.putText(out, text, (cx, y), cv2.FONT_HERSHEY_SIMPLEX, 0.38,
_LABEL_FG, 1, cv2.LINE_AA)
```

Speed and cumulative distance (pitch cm → m): ***`SpeedDistanceEstimator.update`: cm→m, gated path length, rolling-window speed in km/h (§4.7.1–4.7.2, §5.3).***

```python
# Excerpt: Backend/speed_distance/speed_distance_estimator.py


def update(self, frame_num: int, track_id: int, pitch_x_cm: float, pitch_y_cm: float)
-> None:
    pitch_x_m = pitch_x_cm / 100.0
    pitch_y_m = pitch_y_cm / 100.0
```

```
history = self._positions[track_id]


if len(history) > 0:
    _, last_x, last_y = history[-1]
    delta_m = float(np.hypot(pitch_x_m - last_x, pitch_y_m - last_y))
    if delta_m < MAX_PLAUSIBLE_DELTA_M:
        self._total_distance[track_id] += delta_m


history.append((frame_num, pitch_x_m, pitch_y_m))


if len(history) >= 2:
    oldest_frame, oldest_x, oldest_y = history[0]
    newest_frame, newest_x, newest_y = history[-1]
    elapsed_frames = newest_frame - oldest_frame
    if elapsed_frames > 0:
        dist_window_m = float(np.hypot(newest_x - oldest_x, newest_y - oldest_y))
        elapsed_s = elapsed_frames / self.fps
        speed_ms = dist_window_m / elapsed_s
        self._speed_kmh[track_id] = speed_ms * 3.6
else:
    self._speed_kmh[track_id] = 0.0
```

```

**Ball possession (nearest player in metres + grace):** ***Full `BallPossessionAssigner`: nearest-neighbour in metres, radius gate, grace countdown (§5.4).***

```python

# Excerpt: Backend/possession/ball_possession_assigner.py


def assign(self, ball_xy_m, players_xy_m: dict):
    if ball_xy_m is None or len(players_xy_m) == 0:
        return self._handle_no_ball()


    min_dist = float("inf")
    closest_team = None
    for _track_id, (player_xy, team_id) in players_xy_m.items():
        dist = float(np.linalg.norm(ball_xy_m - player_xy))
        if dist < min_dist:
            min_dist = dist
            closest_team = team_id


    if min_dist <= MAX_BALL_PLAYER_DISTANCE_M:
        self._last_team = closest_team
        self._grace_countdown = POSSESSION_GRACE_FRAMES
```
```

```
        self._frame_possession.append(closest_team)
        return closest_team
    return self._handle_no_ball()


def get_percentages(self) -> tuple:
    valid = [t for t in self._frame_possession if t is not None]
    if not valid:
        return 0.0, 0.0
    total = len(valid)
    team_0 = valid.count(0) / total * 100
    team_1 = valid.count(1) / total * 100
    return round(team_0, 1), round(team_1, 1)
```

Eight-view output routing (implementation glue): ***View names written in the analysis pass (cross-reference §5.5).***

```python
# Excerpt: Backend/config.py — OUTPUT_PATHS (uses OUTPUT_DIR)
```

```
OUTPUT_PATHS = {
    "broadcast":       f"{OUTPUT_DIR}/view_1_broadcast.mp4",
    "speed_overlay":   f"{OUTPUT_DIR}/view_2_speed.mp4",
    "distance_overlay": f"{OUTPUT_DIR}/view_3_distance.mp4",
    "combined_overlay": f"{OUTPUT_DIR}/view_4_combined.mp4",
    "radar":           f"{OUTPUT_DIR}/view_5_radar.mp4",
    "radar_speed":     f"{OUTPUT_DIR}/view_6_radar_speed.mp4",
    "radar_distance":  f"{OUTPUT_DIR}/view_7_radar_distance.mp4",
    "radar_combined":  f"{OUTPUT_DIR}/view_8_radar_combined.mp4",
}
```

Renderer registry + per-view `VideoWriter` canvas size (radar 1300×800 vs broadcast native resolution).*

```python
# Excerpt: Backend/main.py


RENDERERS = {
    "broadcast":       BroadcastRenderer(),
    "speed_overlay":   SpeedRenderer(),
    "distance_overlay": DistanceRenderer(),
    "combined_overlay": CombinedRenderer(),
```

```
    "radar":            RadarRenderer(),
    "radar_speed":      RadarSpeedRenderer(),
    "radar_distance":   RadarDistanceRenderer(),
    "radar_combined":   RadarCombinedRenderer(),
}


for view_name, rel_path in OUTPUT_PATHS.items():
    out_path = os.path.join(run_folder, os.path.basename(rel_path))
    if view_name.startswith("radar"):
        w_size = (1300, 800)
    else:
        w_size = (video_info.resolution_wh[0], video_info.resolution_wh[1])
    writers[view_name] = cv2.VideoWriter(out_path, fourcc, fps, w_size)
```

Single-frame multi-view render loop (§5.5 “combined analytics”).

```python
for view_name, renderer in RENDERERS.items():
    try:
        annotated = renderer.render(frame, frame_data)
        writers[view_name].write(annotated)
    except Exception as exc:
```

```
        if frame_num % 100 == 0:
            log(f"  [WARN] {view_name} frame {frame_num}: {exc}")
```

`IDMemory` re-link logic (ByteTrack id → stable id):*Greedy re-attachment with axis limits and stricter radius shortly after loss (§4.4.2 "persistence" beyond ByteTrack buffer).*

```python
# Excerpt: Backend/id_memory/id_memory.py — update() for a new tracker_id


if tracker_id in self.id_map:
    stable_id = self.id_map[tracker_id]
    self.last_positions[stable_id] = position
    self.frames_missing[stable_id] = 0
    self._seen_this_frame.add(stable_id)
    return stable_id


best_sid, best_dist = None, float("inf")
for sid, last_xy in self.last_positions.items():
    if self.frames_missing.get(sid, 0) == 0:
        continue
    if self.frames_missing.get(sid, 0) > self.max_frames_missing:
        continue
```

```
    dx = position[0] - last_xy[0]
    dy = position[1] - last_xy[1]
    if abs(dx) > self.max_x_distance or abs(dy) > self.max_y_distance:
        continue
    dist = (dx * dx + dy * dy) ** 0.5
    missing = self.frames_missing.get(sid, 0)
    allowed_dist = self.max_distance * (0.7 if missing <= 5 else 1.0)
    if dist > allowed_dist:
        continue
    if dist < best_dist:
        best_dist, best_sid = dist, sid

stable_id = best_sid if best_sid is not None else self.next_stable_id
# ... mint next_stable_id, update id_map / last_positions ...
```

Combined broadcast metrics (dual-line label): `*CombinedRenderer`: speed + distance on one HUD block; possession bar (§5.5).*

```python
# Excerpt: Backend/renderers/combined_renderer.py

for category in ("players", "goalkeepers"):
```

```
    for tid, edata in frame_data.get(category, {}).items():
        bbox   = edata["bbox"]
        speed  = edata.get("speed_kmh", 0.0)
        dist_m = edata.get("distance_m", 0.0)
        color  = get_entity_color("player", edata.get("team_id", 0))
        draw_ellipse_marker(out, bbox, color)
        line1 = f"#{tid}"
        line2 = (
            f"{speed:.1f} km/h | {dist_m / 1000:.1f}km"
            if dist_m >= 1000
            else f"{speed:.1f} km/h | {dist_m:.0f}m"
        )
        cx = int((bbox[0] + bbox[2]) / 2)
        y_base = int(bbox[1]) - 8
        self._draw_dual_label(out, line1, line2, (cx - 35, y_base))
```

Tactical radar — Voronoi-style territory blend: ***Per-pixel distance grids to team anchors + `tanh` soft boundary (§5.2 "Voronoi").***

```python
# Excerpt: Backend/renderers/radar_renderer.py — draw_pitch_voronoi_diagram_2()
```

```
y_coords, x_coords = np.indices((scaled_width, scaled_length))


def calculate_distances(xy, x_coordinates, y_coordinates):
    if len(xy) == 0:
        return np.full_like(x_coordinates, 1e9, dtype=np.float32)
    return np.sqrt(
        (xy[:, 0][:, None, None] * scale - x_coordinates) ** 2
        + (xy[:, 1][:, None, None] * scale - y_coordinates) ** 2
    )


min_d1 = np.min(calculate_distances(team_1_xy, x_coords, y_coords), axis=0)
min_d2 = np.min(calculate_distances(team_2_xy, x_coords, y_coords), axis=0)


distance_ratio = min_d2 / np.clip(min_d1 + min_d2, a_min=1e-5, a_max=None)
blend_factor = np.tanh((distance_ratio - 0.5) * 15) * 0.5 + 0.5
# ... blend team colours into pitch interior via cv2.addWeighted ...
```

`FrameData` schema + ball hold (export to renderers): *`frame_data` dict keys + ball interpolation when detection drops (§5.1 ball trajectory).*

```python
# Excerpt: Backend/main.py — frame_data assembly
```

```
frame_data = {
    "frame_num": frame_num,
    "ball": {},
    "players": {},
    "goalkeepers": {},
    "referees": {},
    "possession": {
        "team_id": curr_team,
        "team_0_pct": team_0_pct,
        "team_1_pct": team_1_pct,
    },
}


if len(ball_dets) > 0 and ball_cm:
    _last_ball_canvas = ball_cm[0]
    _last_ball_bbox   = ball_dets.xyxy[0].tolist()
    _ball_missing_cnt = 0
    frame_data["ball"]["bbox"] = _last_ball_bbox
    frame_data["ball"]["canvas_xy"] = _last_ball_canvas
```

```
elif _last_ball_canvas is not None and _ball_missing_cnt < _BALL_HOLD_FRAMES:
    _ball_missing_cnt += 1
    frame_data["ball"]["canvas_xy"] = _last_ball_canvas
    if _last_ball_bbox is not None:
        frame_data["ball"]["bbox"] = _last_ball_bbox


for i in range(len(players_det)):
    sid = int(players_sids[i])
    entry = {"bbox": players_det.xyxy[i].tolist(), "team_id": int(player_team_ids[i])}
    if i < len(players_cm):
        entry["canvas_xy"] = players_cm[i]
        entry["pitch_m"] = np.array(players_cm[i])
    entry["speed_kmh"]  = speed_dist.get_speed(sid)
    entry["distance_m"] = speed_dist.get_distance(sid)
    frame_data["players"][sid] = entry
```

Diagnostics CSV (`frame_diagnostics.csv`): ***Logged homography / entity-count / possession fields for evaluation chapters.***

```python
# Excerpt: Backend/diagnostics/frame_logger.py
```

```
FIELDNAMES = [
    "frame", "h_status", "h_pairs", "h_median_reproj_cm", "h_hist_len",
    "h_frob_delta", "h_spike_rejected",
    "n_non_ball", "n_player", "n_ref", "n_gk",
    "max_pitch_jump_cm", "ball_seen", "poss_team", "team0_pct", "team1_pct",
]
```

```python
# Excerpt: Backend/main.py — payload written each (subsampled) frame

frame_diag.log(frame_num, {
    "h_status": homography.last_status,
    "h_pairs": homography.last_num_pairs,
    "h_median_reproj_cm": round(homography.last_reproj_error, 4),
    "h_hist_len": homography.history_size,
    "h_frob_delta": round(homography.last_h_frob_delta, 6),
    "h_spike_rejected": homography.last_spike_rejected,
    "n_non_ball": len(non_ball),
    "n_player": len(players_det),
    "n_ref": len(referees_det),
```

```
    "n_gk": len(gk_det),
    "max_pitch_jump_cm": round(max_jump_cm, 4),
    "ball_seen": bool(ball_cm and len(ball_cm) > 0),
    "poss_team": curr_team if curr_team is not None else "",
    "team0_pct": round(team_0_pct, 2),
    "team1_pct": round(team_1_pct, 2),
})
```

Pixel → pitch cm projection helper: ***Batch transform used for players, GKs, refs, ball (§4.6.3).***

```python
# Excerpt: Backend/main.py — _transform_points()


def _transform_points(pixel_xy: np.ndarray, transformer) -> list:
    if transformer is None or len(pixel_xy) == 0:
        return []
    cm_pts_array = transformer.transform_points(points=pixel_xy)
    return [tuple(pt) for pt in cm_pts_array]
```
